CCIL: Continuity-Based Corrective Labels for Imitation Learning

Liyiming Ke*, Abhay Deshpande*, Quinn Pfeifer, Yunchu Zhang, Abhishek Gupta, and Siddhartha S. Srinivasa

Abstract

We propose a novel framework to enhance the robustness of imitation learning by generating corrective labels that mitigate compounding errors and disturbances in both simulated and real-world robotics settings. Unlike existing methods that rely on interactive experts or additional datasets, our method, **Continuity-based Corrective Labels for Imitation Learning (CCIL)**, leverages inherent local continuity in environment dynamics. CCIL constructs a locally continuous dynamics model using expert demonstrations and leverages it to generate corrective labels within the vicinity of these demonstrations. Our experiments validate CCIL's efficacy across various simulated robotics tasks, including classic control, drone flight, sensor-based navigation, legged locomotion, and tabletop manipulation. In real-world scenarios, we validate its performance on fine manipulation tasks such as precise grasping and peg insertion, where CCIL notably improves outcomes in contact-rich environments. We demonstrate the value of synthetic corrective labels in low-data regimes and introduce an error-based filtering mechanism to enhance the reliability of synthetic labels. We provide practical insights for implementing CCIL to improve the resilience of various imitation learning state of art methods like diffusion policies and action chunk prediction.

Keywords

Manipulation and Grasping; Deep Learning in Grasping and Manipulation; Imitation Learning; Learning from Demonstration;

1 Introduction

The application of imitation learning in real-world robotics necessitates extensive data and comprehensive coverage (1; 2; 3). However, obtaining such datasets can be prohibitively expensive, particularly in domains that lack a readily available plethora of expert demonstrations. Furthermore, robotic policies can behave unpredictably and dangerously when encountering states not covered in the expert dataset, due to various factors such as sensor noise, stochastic environments, external disturbances, or compounding errors leading to covariate shift (4; 5). For widespread deployment of imitation learning in real-world robotic applications, policies need to ensure their robustness even when encountering unfamiliar states.

Many strategies to address this challenge revolve around augmenting the training dataset. Classic techniques for data augmentation require either an interactive expert (4; 6) or knowledge about the systems' invariances (2; 7; 8). This information is necessary to inform the learning agent how to recover from out-of-distribution states. However, demanding such information can be expensive (8), burdensome (9), or impractical to scale across diverse domains. As a result, behavior cloning, which assumes only access to expert demonstrations, has remained the de-facto solution in many domains (3; 9; 10; 11; 12; 13; 14; 15).

For general applicability, this work focuses on imitation learning (IL) methods that rely *solely on expert demonstrations*. We propose a method to enhance robustness of IL by generating corrective labels for data augmentation. We recognize an under-exploited feature of dynamic systems:

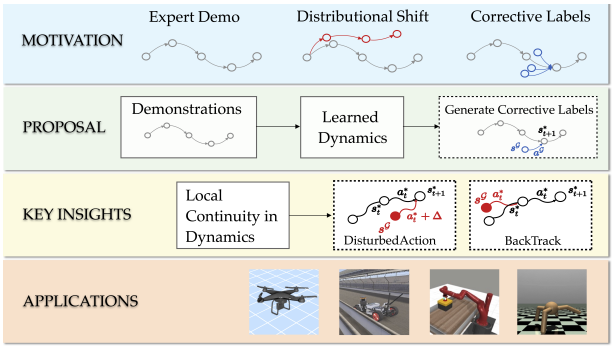


Figure 1. Our proposed framework, CCIL. To enhance the robustness of imitation learning, **CCIL** augments the dataset with synthetic corrective labels. We leverage the local continuity in the dynamics, learn a regularized dynamics function and generate corrective labels *near* the expert data support. We provide theoretical guarantees on the quality of the generated labels.

local continuity. Despite the complex transitions and representations in system dynamics, they need to adhere to the laws of physics and exhibit *some level* of local continuity, where small changes to actions or states result in small changes in transitions. While realistic systems may contain discontinuities in certain regions of the state space, the subset exhibiting local continuity can be a valuable asset.

Armed with this insight, our goal is to synthesize *corrective* labels that guide an agent encountering unfamiliar states back to the distribution of expert states. Leveraging the presence of local continuity makes the synthesis of these corrective labels more tractable. A learned dynamics model with local Lipschitz continuity can navigate an agent from unfamiliar out-of-distribution states to in-distribution expert

University of Washington

Corresponding author:
Abhay Deshpande
Email: abhayd@cs.washington.edu

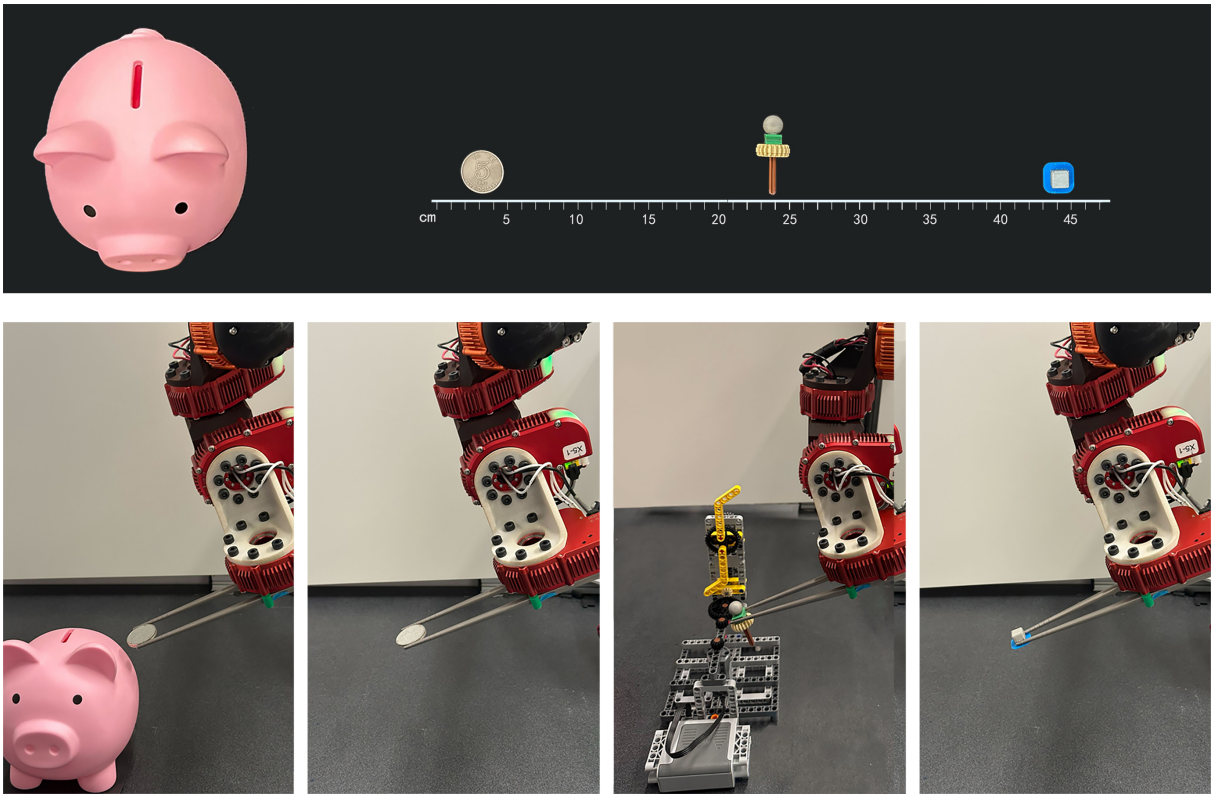


Figure 2. We validates CCIL on real-world fine manipulation tasks with small objects that require sub-milimeter precision: *Grasp&InsertCoin*, *GraspCoin*, *GearInsertion* and *GraspCube*

trajectories, even extending beyond the expert data support. The local Lipschitz continuity allows the model to have bounded error in regions *outside* the expert data support, providing the ability to generate appropriately corrective labels.

We propose a mechanism for learning dynamics models with local continuity from expert data and generating corrective labels. Our practical algorithm, **CCIL**, leverages local **C**ontinuity in dynamics to generate **C**orrective labels for **I**mitation **L**earning. We validate CCIL empirically on a variety of robotic domains in simulation. In summary, our contributions are:

- **Problem Formulation:** We present a formal definition of *corrective labels* to enhance robustness for imitation learning. (Sec. 4.1).

- **Practical Algorithm:** We introduce **CCIL**, a framework for learning dynamics functions, and leveraging local continuity to generate corrective labels. Our method requires only minimal assumptions, primarily relying on availability of expert demonstrations and the presence of local continuity in the dynamics.

- **Theoretical Guarantees:** We showcase how local continuity in dynamic systems would allow extending a learned model's capabilities beyond the expert data. We present practical methods to enforce desired local smoothness while fitting a dynamics function to the data and accommodating discontinuity (Sec. 4.2). We provide a theoretical bound on the quality of the model in this area and the generated labels (Sec. 4.3).

- **Empirical Validation in Simulation:** We conduct experiments over 4 distinct robotic domains across 14 tasks in simulation, ranging from classic control, drone flying, high-dimensional car navigation, legged locomotion and tabletop manipulation. We showcase our proposal's ability to enhance the performance and robustness of imitation learning agents (Sec. 5).

- **In-depth Study of Real-World Applications:** We perform extensive evaluation on a real robotic platform for fine manipulation, demonstrating **CCIL**'s ability to enhance imitation learning under limited data condition in the real world (Sec. 6). We chose three challenging tasks where classical Behavior Cloning struggles even with hundreds of expert trajectories (Fig. 2). Further, we test whether **CCIL** can improve a state-of-the-art imitation learning method, Diffusion Policy (15), and whether it can achieve positive results across varying dataset sizes. We document comprehensive ablation studies to further analyze the impact of design choices in practical implementation of **CCIL**.

This paper unifies and extends two of our previous papers, by Ke et al. (16) which introduced the theoretical framework of CCIL, and by Deshpande et al. (17) which empirically demonstrated its efficacy on real-world fine manipulation tasks when compared against Behavior Cloning (BC). In this paper, we extend CCIL to support *action chunking* (18) and Diffusion Policies (15), and perform an extensive empirical evaluation on real-world manipulation tasks.

2 Related Work

Imitation Learning (IL) Imitation Learning learns a policy from a set of expert demonstrations (19). The simple and practical method Behavior Cloning (BC) formulates IL as a supervised learning problem and remains a strong empirical baseline (10). Recently, a variant of behavior cloning using Diffusion Policy (DP) (15) becomes the new state-of-the-art method and has seen wide adoption in academia and industry (20; 21; 22).

Data Augmentation Imitation Learning has a plethora of data augmentation method in various domains (23; 24). Previous methods for robot manipulation mostly leverage interactive expert (4) or domain-specific property like invariance in the action space (25; 2; 7; 26; 8; 9). Notably, Ke et al. (9) explores noise-based data augmentation for position-control manipulators, but lacks principle for choosing appropriate noise parameters. Block et al. (27) constructs a stabilizing controller around expert demonstrations, conceptually similar to our own approach, but relies on suitably parameterizable low-level stabilizing controllers, which may be difficult to formulate. Park and Wong (28) learns a dynamics model for data augmentation, similar to our approach, but learns an *inverse* dynamics model and lacks theoretical insights. In contrast, our proposal leverages local continuity in the dynamics, is agnostic to domain knowledge, and provides theoretical guarantees on the quality of generated data.

Mitigating Covariate Shift in Imitation Learning. Compounding errors push the agent astray from expert demonstrations. Prior works addressing the covariate shift often request additional information. Methods like DAGGER (4), LOLS (29), DART (6) and AggrevateD (30) use interactive experts, while GAIL (31), SQIL (32) and AIRL (33) sample more transitions in the environment to minimize the divergence of states distribution between the learner and expert (34; 35). Offline Reinforcement Learning methods like MOReL (36) are optimized to mitigate covariate shift but demand access to a ground truth reward function. Reichlin et al. (37) trains a recovery policy to move the agent back to data manifold by assuming that the dynamics is known. MILO (38) learns a dynamics function to mitigate covariate shift but requires access to abundant offline data to learn a high-fidelity dynamics function. In contrast, our proposal is designed to complement existing imitation learning methods, without requiring additional data or feedback. We include MILO and MOReL as baselines in experiments.

Local Lipschitz Continuity in Dynamics. Classical control methods often assume local Lipschitz continuity in the dynamics to guarantee the existence and uniqueness of solutions to differential equations. For example, the widely adopted $\mathcal{C}^2$ assumption in optimal control theory (39) and the popular control framework iLQR (40). This assumption is particularly useful in the context of nonlinear systems and is prevalent in modern robot applications (41; 42; 43). However, most of these methods leveraging dynamics continuity require a pre-specified dynamics model and cost function, and they are rarely seen outside optimal control setting. In contrast, this work focuses on robustifying imitation learning agents by learning a locally continuous dynamics model from data, without requiring a human-specified model or cost function.

Learning Dynamics using Neural Networks. Fitting a dynamics function from data is an active area of research (44; 45; 46; 47; 48). Here we focus on works that do not make assumption on the closed form solutions of the environmental dynamics and instead attempt to learn the dynamics using minimal assumptions via general function approximators like neural networks. For continuous states and transitions, Asadi et al. (49) proved that enforcing Lipschitz continuity in training dynamics model could lower the multi-step prediction errors. Ensuring *local* continuity in the trained dynamics, however, can be challenging. Previous works enforced Lipschitz continuity in training neural networks (50; 51; 52), but did not apply it to dynamics modeling or generating corrective labels. Shi et al. (53) learned smooth dynamics functions via enforcing *global* Lipschitz bounds and demonstrates on the problem of drone landing. Pfrommer et al. (54) learned a smooth model for fictional contacts for manipulation. While this work is not focusing on learning from visual inputs, works such as Khetarpal et al. (55); Zhang et al. (56); Zhu et al. (57) are actively building techniques for learning dynamics models from high-dimensional inputs that could be leveraged in conjunction with the insights from our proposal.

3 Preliminaries

We consider an episodic finite-horizon Markov Decision Process (MDP), $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, f, P_0\}$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, f is the ground truth dynamics function and P_0 is the initial state distribution. A *policy* maps a state to a distribution of actions $\pi \in \Pi : s \to a$. The dynamics function f can be written as a mapping from a state s_t and an action a_t at time t to the change to a new state s_{t+1}: $f(s_t, a_t) = s_{t+1} - s_t$. Following the imitation learning setting, the true dynamics function f and reward are unknown, and we only have transitions drawn from the system dynamics.

We are given a set of demonstrations $\mathcal{D}^*$ as a collection of transition triplets: $\mathcal{D}^* = \{(s_j^*, a_j^*, s_{j+1}^*)\}_j$, where $s_j^*, s_{j+1}^* \in \mathcal{S}, a_j^* \in \mathcal{A}$. We can learn a policy from these traces via supervised learning (behavior cloning) by maximizing the likelihood of the expert actions being produced at the training states: $\arg\max_{\hat{\pi}} \mathbb{E}_{s_j^*, a_j^*, s_{j+1}^* \sim \mathcal{D}^*} \log(\hat{\pi}(a_j^* \mid s_j^*))$. In practice, we can optimize this objective by minimizing the mean-squared error regression using standard stochastic optimization.

In this paper, we consider the local continuity of the dynamics function of the environment, which we formally define as the local Lipschitz continuity.

Definition 3.1. Local Lipschitz Continuity. A function f is *locally Lipschitz continuous* around (s, a) with coefficient K if there exists a neighborhood of (s, a) of size δ such that for every $(\tilde{s}, \tilde{a})$ where $\|(s, a) - (\tilde{s}, \tilde{a})\| \leq \delta$:

$$\|f(s, a) - f(\tilde{s}, \tilde{a})\| \leq K\|(s, a) - (\tilde{s}, \tilde{a})\|. \quad (1)$$

4 Generating Corrective Labels for Robust Imitation Learning Leveraging Continuity in Dynamics

Our objective is to enhance the robustness of imitation learning methods by generating corrective labels that bring an agent from out-of-distribution states back to in-distribution states. We first define the desired *high quality corrective* labels to make imitation learning more robust in Sec. 4.1. We then discuss how these labels can be generated with a known ground truth dynamics of the system (App. A). Without the ground truth dynamics, we discuss how this can be done using a *learned* dynamics model. We present a method to train a locally Lipschitz-bounded dynamics model in Sec. 4.2, and then show how to use such a learned model to generate corrective labels with high confidence (Sec. 4.3), beyond the support of the expert training data. Finally, in Section 4.4, we instantiate these insights into a practical algorithm - **CCIL** that improves the robustness of imitation learning methods with function approximation.

4.1 Corrective Labels Formulation

To robustify imitation learning, we aim to provide corrections to disturbances by bringing the agent back to the "known" expert data distribution, where the policy has been trained and is likely to be successful. We generate state-action-state triplets $(s^{\mathcal{G}}, a^{\mathcal{G}}, s^*)$ that are *corrective*: executing action $a^{\mathcal{G}}$ in state $s^{\mathcal{G}}$ can bring the agent back to a state s^* in the support of the expert data distribution.

Corrective Labels. $(s^{\mathcal{G}}, a^{\mathcal{G}}, s^*)$ is a corrective label triplet with starting state $s^{\mathcal{G}}$, corrective action label $a^{\mathcal{G}}$ and target state (in the expert data) $s^* \in \mathcal{D}^*$ if, with respect to dynamics function f and a small error term ϵ_c,

$$\|[s^{\mathcal{G}} + f(s^{\mathcal{G}}, a^{\mathcal{G}})] - s^*\| \leq \epsilon_c. \tag{2}$$

This definition is trivially satisfied by a given expert demonstration $(s^*_j, a^*_j, s^*_{j+1}) \in \mathcal{D}^*$. We aim to search for a larger set of corrective labels for out-of-distribution states, effectively growing the support of the demonstration data. If a policy has bounded error with respect to the expert on this increased support, it is robust against a larger set of states that it might encounter during execution. However, since the true system dynamics f are unknown, we must use an approximation $\hat{f}$ of the dynamics function derived from finite samples.

Definition 4.1. [High Quality Corrective Labels under Approximate Dynamic Models]. $(s^{\mathcal{G}}, a^{\mathcal{G}}, s^*)$ is a corrective label if $\|[s^{\mathcal{G}} + \hat{f}(s^{\mathcal{G}}, a^{\mathcal{G}})] - s^*\| \leq \epsilon_c$, w.r.t. an approximate dynamics $\hat{f}$ for a target state $s^* \in \mathcal{D}^*$ and a small error ϵ_c. Such a label is "high-quality" if the approximate dynamics function also has bounded error ϵ_{co} w.r.t. the ground truth dynamics function evaluated at the given state-action pair $\|f(s^{\mathcal{G}}, a^{\mathcal{G}}) - \hat{f}(s^{\mathcal{G}}, a^{\mathcal{G}})\| \leq \epsilon_{co}$.

The high-quality corrective labels represent the learned dynamics model's best guess at bringing the agent back into the support of expert data. However, an approximate dynamics model is only reliable in a certain region of the state space, i.e., where the predictions of $\hat{f}$ closely align with the true dynamics of the system, f. When the dynamics model is trained without constraints, such as in a maximum likelihood framework, its performance might significantly degrade when extrapolating beyond the distribution of the training data. To address this challenge, we will first describe how to learn locally continuous dynamics models from data. Subsequently, we will outline how to utilize these models for high-quality corrective label generation beyond the expert data.

4.2 Learning Locally Continuous Dynamics Models from Data

Our core insight for learning dynamics models and using them beyond the training data is to leverage the local continuity in dynamic systems. When the dynamics are locally Lipschitz bounded, small changes in state and/or action yield small changes in the transitions. A dynamics function that exhibit locally continuity would allow us to extrapolate beyond the training data to the states and actions that are in proximity to the expert demonstration - a region in which we can trust the learned model.

We follow the model-based reinforcement learning framework to learn dynamics models from data (58). Essentially, we learn a residual dynamics model $\hat{f}(s^*_t, a^*_t) \rightarrow s^*_{t+1} - s^*_t$ via mean-squared error regression (MSE). We can use any function approximator (e.g., neural network) and write down the learning loss:

$$\arg\min_{\hat{f}} \mathbb{E}_{s^*_j, a^*_j, s^*_{j+1} \sim \mathcal{D}^*} \left[\|\hat{f}(s^*_j, a^*_j) + s^*_j - s^*_{j+1}\| \right] \tag{3}$$

A model trained with the MSE loss alone is not guaranteed to have good extrapolation beyond the training data. Critically, we modify the above learning objective to ensure the learned dynamics model to contain regions that are locally continuous. Previously Shi et al. (53) used spectral normalization to fit dynamics functions that are *globally* Lipschitz bounded. However, most robotics problems involve dynamics models that are hybrids of local continuity and discontinuity: for instance, making and breaking contacts. In face of discontinuity, we present a few practical methods to fit a dynamics model that (1) enforces as much local continuity as permitted by the data and (2) discards the discontinuous regions when subsequently generating corrective labels. For conciseness, we defer the reader to App. C.1 for a complete list of candidate loss functions considered. Here, we show an example of a sampling-based penalty of violation of local Lipschitz constraint.

Local Lipschitz Continuity via Sampling-based Penalty. Following (59), we relax the global Lipschitz constraint by penalizing any violation of the local Lipschitz constraint.

$$\arg\min_{\hat{f}} E_{s^*_j, a^*_j, s^*_{j+1} \sim \mathcal{D}^*} \left[\text{MSE} + \lambda \cdot \mathbb{1}\big(\hat{f}'(s^*_j, a^*_j) > K\big) \right] \tag{4}$$

$$\hat{f}'(s^*_j, a^*_j) \approx \mathbb{E}_{\Delta_s \sim \mathcal{N}} \left[\frac{\|\hat{f}(s^*_j + \Delta_s, a^*_j) - \hat{f}(s^*_j, a^*_j)\|}{\|\Delta_s\|} \right] \tag{5}$$

We use samples to estimate the local Lipschtiz continuity $\hat{f}'(s_j^*, a_j^*)$, by perturbing the data points with small sampled Gaussian noises $\Delta_s \sim \mathcal{N}$. Alternatively, one can compute the Jacobian matrix to estimate the local Lipschitz continuity. The penalty term weighted by λ enforces the local Lipschitz constraint at the specific expert datapoint. Doing so ensures that the approximate model is mostly K Lipschitz-bounded while being predictive of the transitions in the expert data. This approximate dynamics model can then be used to generate corrective labels.

4.3 Generating High-Quality Corrective Labels

We employ learned dynamics models to generate corrective labels, leveraging local continuity to ensure the generated labels have bounded error in proximity to the expert data's support. When the local dynamics function is bounded by a Lipschitz constant w.r.t. the state-action space, we can (1) perturb the dynamics function by introducing slight variations in either state or action and (2) quantify the prediction error from the dynamics model based on the Lipschitz constant. We outline two techniques, BackTrack label and DisturbedAction label, to generate corrective labels (Fig. 3).

Technique 1: BackTrack. Assuming local Lipschitz continuity w.r.t. states, given expert state-action pair s_t^*, a_t^*, we propose to find a different state $s_{t-1}^{\mathcal{G}}$ that can arrive at s_t^* using action a_t^*. To do so, we optimize for a state $s_{t-1}^{\mathcal{G}}$ such that:

$$s_{t-1}^{\mathcal{G}} + \hat{f}(s_{t-1}^{\mathcal{G}}, a_t^*) = s_t^* \tag{6}$$

The quality of the generated label $(s_{t-1}^{\mathcal{G}}, a_t^*, s_t^*)$ is bounded and we present the proof in Appendix B.1.

Theorem 4.2. *When the dynamics model has a training error of ϵ on the specific data point, under the assumption that the dynamics models f and $\hat{f}$ are locally Lipschitz continuous w.r.t. state with Lipschitz constant K_1 and K_2 respectively, if $s_t^{\mathcal{G}}$ is in the neighborhood of local continuity, then*

$$\left\| f\left(s_t^{\mathcal{G}}, a_{t+1}^*\right) - \hat{f}\left(s_t^{\mathcal{G}}, a_{t+1}^*\right) \right\| \leq \epsilon + (K_1 + K_2) \left\| s_t^{\mathcal{G}} - s_{t+1}^* \right\|. \tag{7}$$

Technique 2: DisturbedAction. Assuming local Lipschitz continuity w.r.t. state-action pairs, given an expert tuple $(s_t^*, a_t^*, s_{t+1}^*)$, intuitively there are states near s_t^* that can transition to s_{t+1}^* by taking an action $a^{\mathcal{G}}$ similar to a_t^*. We can sample $a_t^{\mathcal{G}} = a_t^* + \Delta$ with $\Delta \sim \mathcal{N}$, and then solve for the state $s_t^{\mathcal{G}}$ that transitions to s_{t+1}^* as follows:

$$s_t^{\mathcal{G}} + \hat{f}(s_t^{\mathcal{G}}, a_t^* + \Delta) - s_{t+1}^* = 0. \tag{8}$$

For every data point, we can obtain a set of labels $(s_t^{\mathcal{G}}, a_t^* + \Delta)$ by randomly sampling the noise vector Δ. We show that the quality of the labels is bounded (proof in App. B.2).

Theorem 4.3. *Given the dynamics function f is locally Lipschitz continuous w.r.t. states and actions, with Lipschitz constants K_S and K_A respectively, and $(s_t^{\mathcal{G}}, a_t^* + \Delta)$ is in the neighborhood of local continuity for $\Delta \sim \mathcal{N}$, if $s_{t+1}^* - \hat{f}(s_t^{\mathcal{G}}, a_t^* + \Delta) - s_t^{\mathcal{G}} = 0$, then*

$$||f(s_t^{\mathcal{G}}, a_t^* + \Delta) - \hat{f}(s_t^{\mathcal{G}}, a_t^* + \Delta)|| \leq K_A||\Delta|| + (1 + K_S)||\epsilon||. \tag{9}$$

Solving the Root-Finding Equation in Generating Labels Both our techniques need to solve root-finding equations (Eq. 6 and 8). This suggests an optimization problem: $\arg\min_{s^{\mathcal{G}}} ||s^{\mathcal{G}} + \hat{f}(s^{\mathcal{G}}, a^{\mathcal{G}}) - s^*||$ given $\hat{f}$ and $a^{\mathcal{G}}, s^*$. There are multiple ways to solve this optimization problem (e.g., gradient descent or Newton's method). For simplicity, we choose a fast-to-compute and conceptually simple solver, Backward Euler, widely adopted in modern simulators (60). It lets us use the gradient of the next state to recover the earlier state. To generate data given $a^{\mathcal{G}}, s^*$, Backward Euler solves a surrogate equation iteratively: $s^{\mathcal{G}} \leftarrow s^* - \hat{f}(s^*, a^{\mathcal{G}})$.

Label Error. Theorems 4.2 and 4.3 show that the error of the labels generated with these techniques are bounded. If we can compute this bound, we can individually determine the quality of each generated label. To do so, we first consider how to compute the local Lipschitz coefficient of the learned dynamics model.

Remark 4.4. Consider a function $f(s, a)$ that is locally Lipschitz-continuous around (s, a) with coefficient K. Letting J_f notate the jacobian of f, then:

$$K \approx \|J_f(s, a)\|_2 \tag{10}$$

Note that we can easily compute the jacobian of our learned dynamics using auto-differentiation libraries. Then, with some suitable approximations, we can use our derived error bounds to calculate the error of each generated label.

Definition 4.5. BackTrack Label Error. Consider Theorem 4.2. We can assume $\epsilon \approx 0$ by only generating labels for data points with low model error, and also that $K_2 \propto K_1$. We define the BackTrack label error to be

$$\mathcal{E}_{\text{BT}} = \|J_{\hat{f}}(s_t^*, a_t^*)\|_2 \cdot \|s_t^{\mathcal{G}} - s_t^*\| \tag{11}$$

Definition 4.6. DisturbedAction Label Error. Consider Theorem 4.3. We can assume $K_A \propto \|J_{\hat{f}}^A(s_t^*, a_t^*)\|_2$ and $K_S \propto \|J_{\hat{f}}^S(s_t^*, a_t^*)\|_2$, where $J_{\hat{f}}^A$ and $J_{\hat{f}}^S$ are the Jacobians of $\hat{f}$ with respect to actions and states, respectively. We define the DisturbedAction label error to be

$$\mathcal{E}_{\text{DA}} = \|J_{\hat{f}}^A(s_t^*, a_t^*)\|_2 \cdot \|\Delta\| + (1 + \|J_{\hat{f}}^S(s_t^*, a_t^*)\|_2) \cdot \|s_t^{\mathcal{G}} - s_t^*\| \tag{12}$$

Label Filtering. Now that we can calculate the quality of each generated label, we can reject labels that result in a large error bound, as defined by Definitions 4.5 and 4.6. In practice, we calculate the label error for all generated labels, and filter out some fraction of the labels with highest associated error. By setting the label error filtering threshold as some quantile, we can directly control the size of the error bound.

4.4 CCIL: Continuity-based Corrective Labels for Imitation Learning

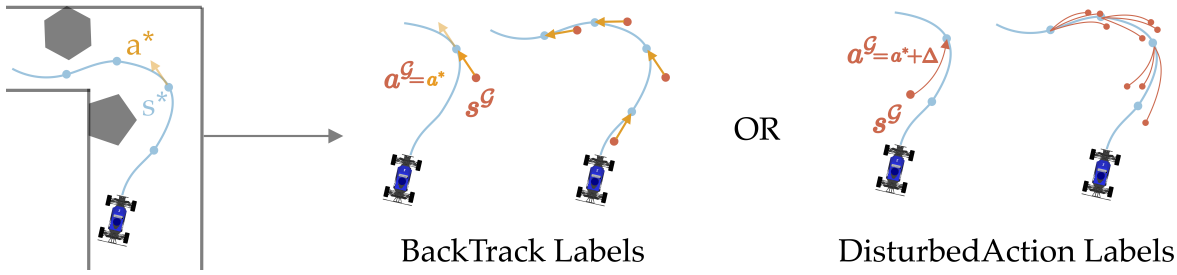


Figure 3. Generating corrective labels from demonstration. Given expert demonstration state s^* and action a^* that arrives at s^*_{next}. BackTrack Labels: search for an alternative starting state $s^{\mathcal{G}}$ that can arrive at expert state s^* if it executes expert action a^*. DisturbedAction Labels: search for an alternative starting state $s^{\mathcal{G}}$ that would arrive at s^*_{next} if it executes a slightly disturbed expert action $a^{\mathcal{G}} = a^*_t + \Delta$, where Δ is a sampled noise.

Algorithm 1: CCIL: **C**ontinuity-based data augmentation for **C**orrective labels for **I**mitation **L**earning

Input: $\mathcal{D}^* = (s^*_i, a^*_i, s^*_{i+1})$
Initialize: $\mathcal{D}^{\mathcal{G}} \leftarrow \varnothing$
LearnDynamics $\hat{f}$
for $i = 1..n$ **do**
  $(s^{\mathcal{G}}_i, a^{\mathcal{G}}_i), \mathcal{E} \leftarrow$ GenLabels $(s^*_i, a^*_i, s^*_{i+1})$
  if $\mathcal{E} < \epsilon$ **then**
    $\mathcal{D}^{\mathcal{G}} \leftarrow \mathcal{D}^{\mathcal{G}} \cup (s^{\mathcal{G}}_i, a^{\mathcal{G}}_i)$
  end if
end for
$\mathcal{D} \leftarrow \mathcal{D}^* \cup \mathcal{D}^{\mathcal{G}}$
// Learn Policy
$\pi \leftarrow \arg\min_\pi \mathbb{E}_{(s_i,a_i)\sim\mathcal{D}} [L(a_i, \pi(s_i))]$
Function LearnDynamics $\hat{f}$
  Optimize a chosen objective from Sec. 4.2
Function GenLabels(BackTrack)
  $a^{\mathcal{G}}_i \leftarrow a^*_i$
  $s^{\mathcal{G}}_i \leftarrow \arg\min_{s^{\mathcal{G}}_i} ||s^{\mathcal{G}}_i + \hat{f}(s^{\mathcal{G}}_i, a^{\mathcal{G}}_i) - s^*_i||$
  $\mathcal{E} \leftarrow \|J_{\hat{f}}(s^*_i, a^*_i)\|_2 \cdot \|s^{\mathcal{G}}_i - s^*_i\|$
Function GenLabels(DisturbedAction)
  $a^{\mathcal{G}}_i \leftarrow a^*_i + \Delta, \Delta \sim \mathcal{N}(0, \Sigma)$
  $s^{\mathcal{G}}_i \leftarrow \arg\min_{s^{\mathcal{G}}_i} ||s^{\mathcal{G}}_i + \hat{f}(s^{\mathcal{G}}_i, a^{\mathcal{G}}) - s^*_{i+1}||$
  $\mathcal{E} \leftarrow \|J^A_{\hat{f}}(s^*_i, a^*_i)\|_2 \cdot \|\Delta\|$
    $+(1 + \|J^S_{\hat{f}}(s^*_i, a^*_i)\|_2) \cdot \|s^{\mathcal{G}}_i - s^*_i\|$

We instantiate a practical version of our proposal, **CCIL** (**C**ontinuity-based data augmentation to generate **C**orrective labels for **I**mitation **L**earning), as shown in Alg 1, with three steps: (1) fit an approximate dynamics function that exhibit local continuity, (2) generate corrective labels using the learned dynamics, and (3) use rejection sampling to select labels with the desired error bounds. We highlight the difference in our two techniques in color. To use the generated data $D^{\mathcal{G}}$, we simply combine it with the expert data and train policies using standard behavior cloning on the augmented dataset. We evaluate the augmented agent to empirically test whether training with synthetic corrective labels would help imitation learning.

5 Simulation Validation

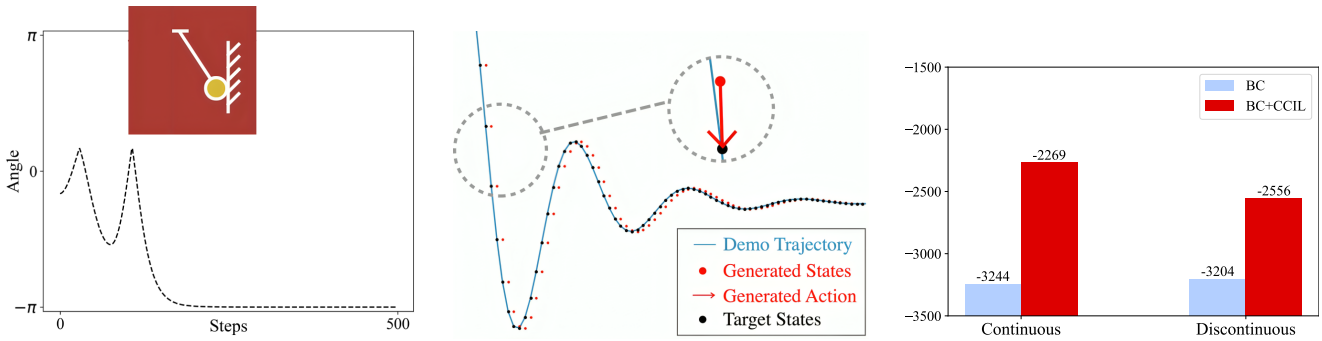
As a warm-up to real-world robot experiments, we first evaluate CCIL over 4 simulated robotic domains of 14 common tasks to answer the following questions:
Q1. Can we empirically verify the theoretical contributions on the quality of the generated labels;
Q2. How well does CCIL handle discontinuities in the environmental dynamics;
Q3. How would training with CCIL-generated labels affect imitation learning agents' performance.

We compare Behavior Cloning (BC) (10) trained with labels generated by CCIL against several baselines: vanilla BC, NoisyBC (9), MILO (38), and MOREL (36). Notably, all other baselines rely on *additional* assumptions that CCIL does not require. For instance, NoisyBC augments imitation learning data through noise injection and assumes the robot operates under position control. MILO learns a dynamic model from demonstrations for data augmentation but requires a large dataset with broad coverage over the state action space. MOReL learns a dynamics function but necessitates access to the reward function. We observe limited performance with baselines when their assumptions are inapplicable. Operating without these assumptions, CCIL achieves competitive or superior performance compared to these methods in various settings. Implementation details for reproducing the experiments are provided in Appendix D. We employ the same model architecture across all methods and evaluate performance over 10 random seeds.

5.1 *Analysis on Classic Control of Pendulum and Discontinuous Pendulum*

We consider the classic control task, the pendulum, and also consider a variant, The Discontinuous Pendulum, which adds two wall to obstruct the pendulum and cause it to bounce back, as shown in Fig. 4(a).

Verifying the quality of the generated labels (Q1). We visualize a subset of generated labels in Fig. 4(b). Note that the generated states (red) are outside the expert support (blue) and that the generated actions are torque control signals, which are challenging for invariance-based data augmentation methods (e.g., NoisyBC) to generate. To quantify the quality of the generated labels, we use the fact that ground truth dynamics can be analytically computed for this domain and measure how closely our labels can bring the agent to the expert. We computed the empirical error using



(a) The Discontinuous Pendulum **(b)** Example generated labels **(c)** Performance Comparison

Figure 4. Evaluation on the Pendulum and Discontinuous Pendulum Task.

the average L2 norm distance and obtained 0.02367, which validates our theoretical bound of 0.065: $K_1|\Delta| + (1 + K_2)|s_t^* - s_t^G| = 12 * 0.0001 + 13 * 0.005$ (Equation 7).

Verifying the method's resilience to presence of discontinuity (Q2) To highlight how local discontinuity in the dynamics affects CCIL, we compare BC+CCIL's performance in the continuous and discontinuous pendulum in Fig. 4(c): CCIL improved behavior cloning performance even in the presence of discontinuity, albeit with a smaller margin for the discontinuous Pendulum.

CCIL improved behavior cloning agent (Q3). For both the continuous and discontinuous Pendulum, training BC with CCIL-generated labels yielded a large performance boost (Fig. 4(c)).

5.2 *Driving with Lidar: High-dimensional state input*

We compare all agents on the F1tenth racing car simulator Fig. 5(a), which employs a high-dimensional, LiDAR sensor as input, Fig. 5(b). In this task, an agent must drive around the track as quickly as possible without crashing. We generate the expert demonstration using a scripted policy that has access to the ground truth state. We evaluate each imitation learning agent over 100 trajectories. At the beginning of each trajectory, the car is placed at a random location on the track. It needs to use the LiDAR input to decide on speed and steering, earning scores for driving faster or failing by crashing.

CCIL can improve the performance of behavior cloning for high-dimensional state inputs (Q2, Q3). Table. 5(c) shows that CCIL demonstrated an empirical advantage over all other agents, achieving a higher success rate and better score while having fewer crashes. Noticeably, LiDaR inputs contain highly-complicated discontinuities and impose challenges to other model-based methods that trust the learned dynamics model everywhere (Morel and MILO), while CCIL could reliably generate corrective labels in a trust region and reject samples with low confidence.

5.3 *Drone Navigation: High-Frequency Control Task and Sensitivity to Noises*

Drone navigation is a high-frequency control task and can be very sensitive to noise, making it an appropriate testbed for the robustness of imitation learning. We use an open-source quadcopter simulator, gym-pybullet-drone (61) and consider three proposed tasks: hover, circle, fly-through-gate, as shown in Fig. 6(a). In each task, the drone will take its current position, quaternion, angular orientation, and angular velocity as observations and then generate PWM control signals for each motor. The reward is defined by how well it follows the desired trajectory. Following Shi et al. (53), we inject observation and action noises during evaluation to highlight the robustness of the learner agent.

CCIL improved performance and robustness to noise when applied to BC (Q3). BC+CCIL outperformed the other listed baselines.

5.4 *Locomotion and Manipulation: Diverse Scenes with Varying Discontinuity*

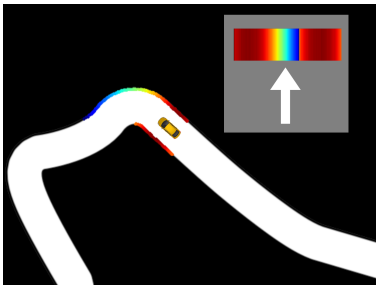
Manipulation and locomotion tasks commonly involve complex forms of contacts, raising considerable challenges for learning dynamics models and for our proposal. We evaluate the applicability of CCIL in such domains, considering 4 tasks from MuJoCo locomotion suite: Hopper, Walker2D, Ant, HalfCheetah and 4 tasks from the MetaWorld manipulation suites: CoffeePull, ButtonPress, CoffeePush, DrawerClose. During evaluation, we add a small amount of randomly sampled Gaussian noise to the sensor (observation state) and the actuator (action) to simulate real-world conditions of robotics controllers and to test the robustness of the agents.

CCIL boosts behavior cloning or at least achieves comparable performance even when varying form of discontinuity is present (Q2, Q3). Across all tasks, BC+CCIL at least achieves comparable results to vanilla behavior cloning, shown in Table. 1, indicating that it's an effective augmentation to behavior cloning without necessitating substantial extra assumptions. On 4 out of 8 tasks considered (Hopper, Walker, Halfcheetah, CoffeePull), BC+CCIL outperforms all baselines. On the rest 4 tasks, BC+CCIL achieved a tied 1st or the 2nd place. Note that MILL and NoiseBC achieved the best performance on 1 and 2 tasks, respectively, which can be attributed to their additional assumptions regarding data coverage and position control.

5.5 *Summary*

We conducted extensive experiments to address three research queries. For **Q1**, we verified the proposed theoretical bound on the quality of the generated labels on the Pendulum task. For **Q2**, we tested that CCIL is robust to environmental discontinuities, improving behavior cloning

(a) F1tenth

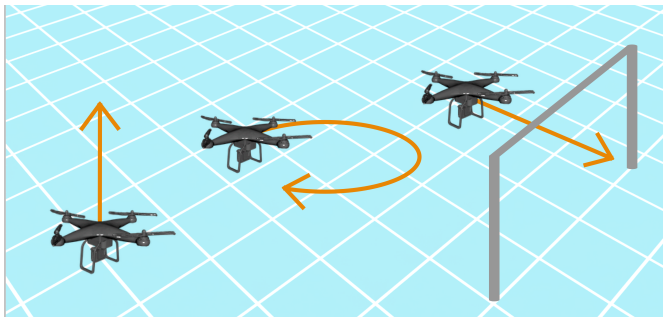


(b) LiDAR POV

Method	Succ. Rate	Avg. Score
Expert	100.0%	1.00
Morel	0.0%	0.001 ± 0.001
MILO	0.0%	0.21 ± 0.003
BC	31.9%	0.58 ± 0.25
NoiseBC	39.3%	0.62 ± 0.28
BC+CCIL	**89.6%**	**0.95** ± 0.08

(c) Performance on F1Tenth Racing

Figure 5. Evaluation on a 2D racing task with LiDAR state.



(a) Drone Flying Tasks

Method	Hover	Circle	FlyThrough
Expert	-1104	-10	-4351
BC	-1.08E8	-9.56E7	-1.06E8
NoisyBC	-1.13E8	-9.88E7	-1.07E8
Morel	-1.25E8	-1.24E8	-1.25E8
MILO	-1.26E8	-1.25E8	-1.25E8
BC+CCIL	-0.77E8	-3.49E7	-0.41E8

(b) Performance on Drone Flying

Figure 6. Evaluation on various drone tasks tasks.

Table 1. Evaluation results for Mujoco and Metaworld tasks with noise disturbances. We list the expert scores in a noise-free setting for reference. BC+CCIL is the leading agent on 4 out of 8 tasks (Hopper, Walker, HalfCheetah, CoffeePull) and achieved a tied 1st or the 2nd place on all the other tasks. Comparing impact of CCIL on BC: across all tasks, CCIL boosts the performance of behavior cloning or at least achieves comparable performance.

	Mujoco				**Metaworld**			
	Hopper	Walker	Ant	Halfcheetah	CoffeePull	ButtonPress	CoffeePush	DrawerClose
Expert	3234.30	4592.30	3879.70	12135.00	4409.95	3895.82	4488.29	4329.34
BC	1983.98 ± 672.66	1922.55 ± 1410.09	2965.20 ± 202.71	8309.31 ± 795.30	3552.59 ±233.41	**3693.02** ± 104.99	1288.19± 746.37	3247.06 ± 468.73
Morel	152.19±34.12	70.27 ± 3.59	1000.77 ± 15.21	-2.24 ± 0.02	18.78±0.09	14.85±17.08	18.66± 0.02	1222.23± 1241.47
MILO	566.98±100.32	526.72±127.99	1006.53±160.43	151.08±117.06	232.49± 110.44	986.46±105.79	230.62±19.37	**4621.11**±39.68
NoiseBC	1563.56 ± 1012.02	2893.21 ± 1076.89	**3776.65** ± 442.13	8468.98 ±738.83	3072.86 ± 785.91	**3663.44**±63.10	**2551.11**± 857.79	4226.71± 18.90
BC+CCIL	**2784.45** ± 188.56	**3724.89** ± 558.01	3546.32 ± 338.84	**9057.13** ± 425.47	**3949.54** ± 252.34	**3827.60**±24.37	**2551.18**± 770.02	4187.90± 44.40

in both continuous (Pendulum, Drone) and discontinuous scenarios (Driving, MuJoCo). For **Q3**, BC+CCIL emerged as the best agent on 10 out of 14 tasks. In the rest of tasks, it achieved comparable performance to vanilla behavior cloning, demonstrating its cost-effectiveness and efficiency as an alternative approach without requiring significant additional assumptions.

6 Real-World Experiment Design

6.1 Motivation

We see that the CCIL framework shows notable success in various simulation domains. However, its application to real-world scenarios hinges on several critical questions:

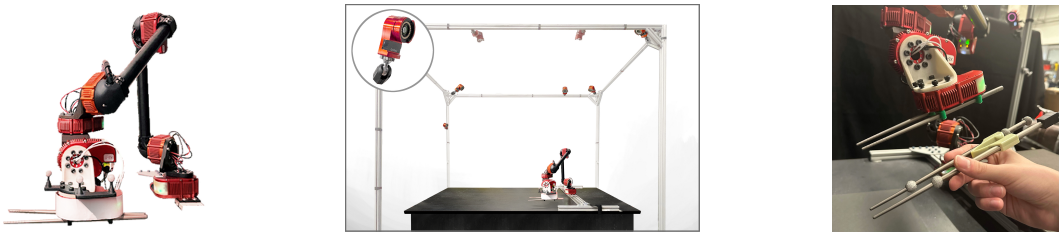
Local Continuity in Dynamics and Real-World Application CCIL relies on local Lipschitz continuity in system dynamics, yet real-world robotic tasks often involve discontinuities due to physical contact. Can CCIL still enhance imitation learning agent performance amidst such challenges?

Data Scarcity and Augmentation Impact Limited real-world robot demonstrations highlight the importance of efficient data augmentation. While abundant data typically guarantees better outcomes, the critical question lies in the low data regime: How effectively can CCIL address data scarcity, and what data volume is required to observe a tangible impact?

Sensitivity to Hyperparameters Tuning imitation learning algorithms on physical robots is logistically challenging. Direct execution on robots, while being the most reliable evaluation method, risks unpredictable behaviors and necessitates numerous real-world trials to achieve statistical significance. How sensitive is CCIL to hyperparameter variations? Can we offer guidelines for its real-world application to mitigate the high evaluation and tuning costs?

6.2 Hardware

To answer these questions, we conduct experiments on a fine manipulator platform introduced in Ke et al. (9) and shown in Fig. 7(a). The robot has 7 joints and is equipped with a pair of chopsticks as its end-effector. It runs a hierarchical controller, where the high-level command consists of a 6-DOF end-effector pose target plus 1-DOF for the last joint to open or close the gripper chopsticks. A policy sends high-level commands at 20Hz. The robot computes inverse kinematics to send each joint a low-level positional



(a) Robot Hardware **(b)** Motion Capture System **(c)** Teleoperation Interface

Figure 7. System overview: (a) shows our HEBI-based 7-DOF robot with a chopstick end-effector, (b) shows our OptiTrack motion-capture cage for perception, and (c) shows teleoperation mimicking leader chopsticks tracked using the motion-capture cage.

command which is tracked by a positional PID controller running at 1KHz.

For perception, we use a motion capture system, pictured in Fig. 7(b). As in Ke et al. (62), we use the motion capture system to collect human teleoperation demonstrations. Specifically, the human expert uses "leader chopsticks" tracked with motion capture markers. The robot mimics the leader chopsticks' pose using inverse kinematics, and in this way is directed by the expert to complete the task (Fig. 7(c)).

6.3 Tasks and Data Collection

We focus on fine manipulation tasks (Fig. 2) that require a millimeter level of precision.

- *GraspCube* - Grasp and lift a tiny 1cm cube above the table. Despite being the easiest of all three tasks, the task is non-trivial with scarce-data (e.g., 100 trajectories).
- *GearInsertion* - Inspired by industrial assembly (63), we design a mini-gear insertion task. The agent needs to insert a Lego gear into a hole on a board. Proper insertion leaves a gap less than 0.2 mm.
- *GraspCoin* - Grasp and lift a metal coin lying flat on the table. The coin's round shape and slippery texture makes it hard even for human experts to pick up using chopsticks.
- *PiggyCoinInsertion* - Grasp and lift a metal coin and insert into a PiggyBank. It requires both precise grasp and insertion.

For all tasks, we vary the initial positions of the object across a region about size 25 cm x 25 cm. To collect demonstrations, we use a mix of heuristic controllers and human teleoperation. Note that the heuristic controllers did not have perfect success rates and we filtered out failed demonstrations. For *GraspCube* and *GearInsertion*, we designed heuristic controllers and collected 500 trajectories for *GraspCube* and 100 trajectories for *GearInsertion*. For *GraspCoin*, we instead used successful demonstrations from expert teleoperation. Due to the difficulty of the task, we only obtained 200 trajectories.

6.4 Hypotheses

We explore the following hypotheses:

- **H1:** CCIL can improve imitation learning policies in real-world fine-manipulation tasks. This improvement is statistically significant, especially in low-data regimes.
- **H2:** Real-world tasks with complex contacts and discontinuities in dynamics can still exhibit local Lipschitz continuity to justify CCIL's assumptions.
- **H3:** The performance of CCIL is highly sensitive to the label rejection hyperparameter, which determines the acceptable error bound for generated labels.
- **H4:** The performance of CCIL exhibits relative robustness to variations in the local Lipschitz constraint enforced during the training of the dynamics model.

For **H.1**, we compare the success rate of a BC agent trained with and without CCIL-generated labels. We also compare the success rate of the state-of-the-art diffusion policy trained with and without CCIL-generated labels. We vary the amount of demonstration data to investigate the impact of data quantity on these methods.

For **H.2**, we measure the local Lipschitz continuity of the trained dynamics model for tasks with complex discontinuity. We also vary the hyperparameter of the upper bound on the local Lipschitz constant.

For **H.3** and **H.4**, we run ablation studies varying the hyper-parameter of label rejection threshold and Lipschitz constraint to investigate (1) their impact on the success rate of CCIL and (2) the interplay between these hyper-parameters.

6.5 Training

For each task we train two behavior cloning agents, with and without CCIL-generated labels. The training follows Alg. 1 but we formulate the action loss for our hardware and detail the parameter tuning procedure.

Loss Formulation To train a policy using behavior cloning as described in Alg. 1, we need to design an action loss $L(a^*, \hat{a})$. Following (9), we denote the action a for our hardware as (x, q, c) representing the end effector xyz location, orientation, and chopstick angle. Given the target action $a^* = (x, q, c)$ and the predicted action $\hat{a} = (\hat{x}, \hat{q}, \hat{c})$, the action loss is:

$$L(a, \hat{a}) = \alpha_1 \|x - \hat{x}\|^2 + \alpha_2 \theta^2 + \alpha_3 (c - \hat{c})^2 \qquad (13)$$

Where θ is the angle between the orientations q and $\hat{q}$, and $\alpha_1, \alpha_2, \alpha_3$ weight each component of the loss function and resolve units. In our experiments, we use $\alpha_1 = 10, \alpha_2 = 1, \alpha_3 = 10$.

Parameter Tuning Applying CCIL introduces two key parameters: the Lipschitz constraint for training the dynamics model and a threshold for filtering the generated labels. The Lipschitz constraint, K (Eq. 4), upper bounds the Lipschitz continuity of the learned dynamics model. A loose upper bound makes the training process easier and more likely to yield low training error. Conversely, a tighter bound is more likely to yield a dynamics model with lower Lipschitz coefficients, enabling CCIL to generate higher-quality corrective labels. The label error bound is defined in Definitions 4.5 and 4.6, and the threshold σ controls the acceptable bound.

To set the Lipschitz constraint K, we first train an unconstrained model and analyze the distribution of local Lipschitz coefficients of the learned model across the training data. If the distribution of coefficient suggests that the learned model has limited local Lipschitz continuity, we choose a tighter Lipschitz constraint.

To set the label error quantile, we first generate corrective labels without filtering and analyze the distribution of label errors. Based on the distribution, we choose a label rejection quantile $\sigma \in [0, 1]$ (where 1 means to filter out all generated labels) that filters out outliers or long tails. Section 7.3 explores this parameter in depth.

Label Generation for Action Chunking Recent imitation learning methods (15) indicate that outputting a chunk of actions instead of a single action can help policy performance. To support these state-of-the-art methods, we must adapt CCIL to support mapping a state to not just one action but a series of actions ("chunk"). Concretely, when we generate a single action label, we place it at the start of a chunk. i.e., we replace the first action in the demonstrated action chunk with our synthetically generated label. Complete details for action chunk generation are outlined in App. C.3, and we test the performance when training with these labels in Section 7.1.

6.6 Evaluation

We test each agent's success rate by conducting 48 trials per agent to establish statistical significance. To ensure fairness, for each task we select 16 fixed initial conditions (i.e., for grasping agents, we place the object to grasp at 16 chosen positions spread across the workplace). For each initial condition, we test the learned policy for 3 trials. For each trial, we denote the success as a binary variable. To report the statistical significance of the empirical results, we compare the success rates between the two policies by performing two-proportion z-tests.

7 Results

Across three tasks, we trained 36 agents and tested their success rate in the real world (Fig. 8(a)). For ablation, we further evaluated 33 agents on the GraspCube task.

7.1 Corrective labels' improvement to imitation learning

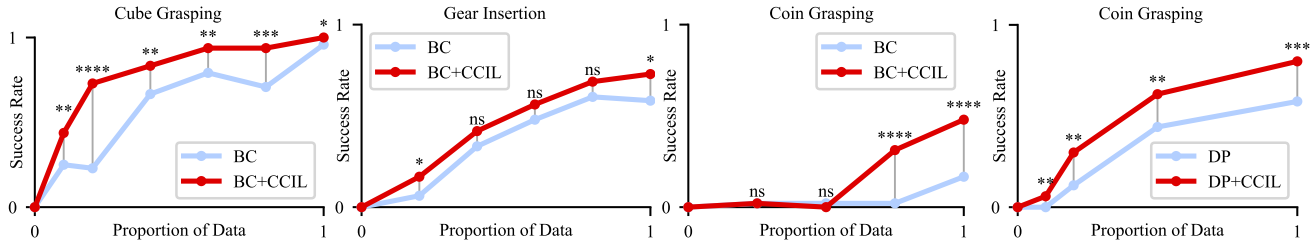
H.1 investigates whether CCIL can improve imitation learning performance in real-world fine-manipulation tasks. These tasks involve complex contact dynamics between objects, end-effectors, and the workspace, making it unclear if CCIL's assumptions hold.

CCIL increases the performance of vanilla behavior cloning. Fig. 8(a) shows that training with labels generated by CCIL generally improves the performance of BC over all three tasks, as measured by success rate. We validated the statistical significant of the improvement. The performance boost is significant at the $p < 0.05$ level for the cube grasping and coin grasping tasks.

CCIL boost is significant in the low-data regime. The empirical performance gain from applying CCIL is statistically significant in low-data regimes, as shown in Fig. 8(a). CCIL could boost BC performance from 23% to 73% (using 20% of GraspCube trajectories), from 6% to 17% (using 20% of GearInsertion trajectories) or 58% to 73% (using 100% of GearInsertion trajectories) and from 17% to 48% (using 100% GraspCoin trajectories). These limited-data regimes face significant challenges from covariate shift due to limited data support from expert demonstrations. CCIL demonstrated promising results to alleviate this problem.

CCIL brings performance increase to diffusion policy We ablate the impact of CCIL applied to the state of the art Diffusion Policy (DP). Due to diffusion policies' sample efficiency, we use less data in training than before and conducted 45 evaluation trials for any agent. For the cube task, we used 5 demo episodes where the baseline DP achieved 62% success rate under test-time disturbance. Applying CCIL boosted its performance to 82.2% despite test-time disturbance. For the hardest task, GraspCoin, we used 50 demonstration episodes for training. Fig. 8(b) demonstrates that CCIL provides a notable performance boost to diffusion policy (DP) on the hardest GraspCoin task. With only 5 demonstrations, CCIL improves the success rate from 0% (0/50) to 6% (3/50). Using 10 demonstrations, DP+CCIL achieves a success rate of 30% (15/50) compared to 12% (6/50) for DP alone. This trend continues with 25 demonstrations (62% vs. 44%) and 50 demonstrations (80% vs. 58%). These results highlight CCIL's effectiveness in mitigating the challenges of covariate shift and limited data support for multiple policy classes, including diffusion-based learning.

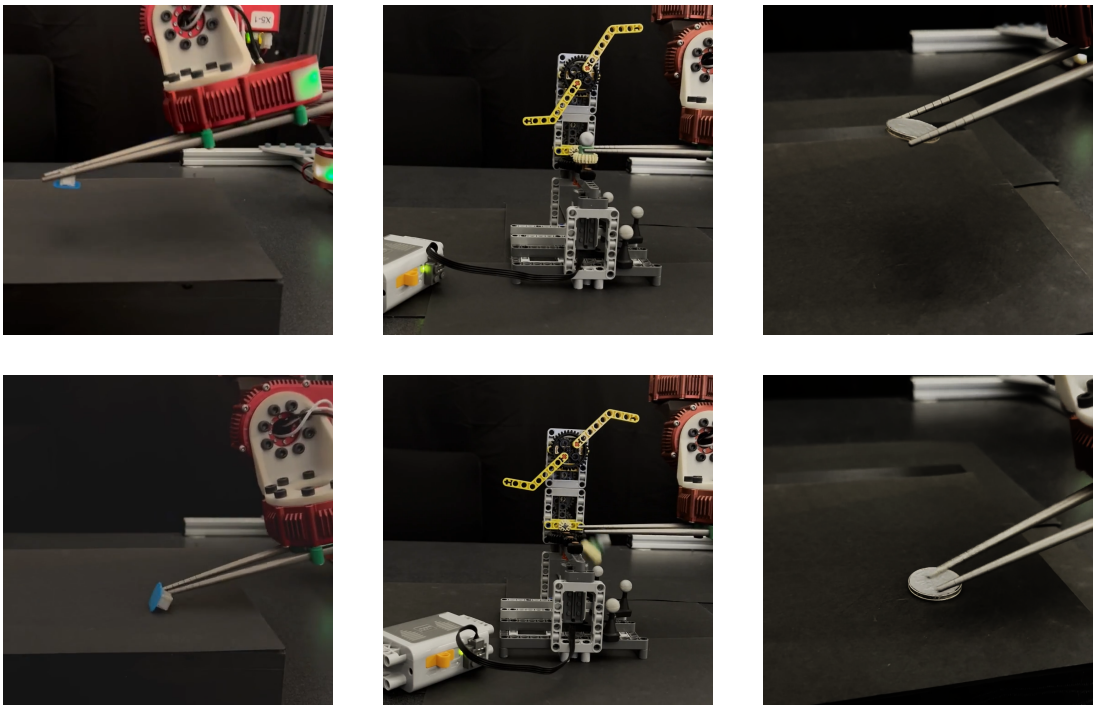
Demonstration on a More Challenging Task Building on the success achieved with *GraspCoin* and *LegoInsert*, we applied CCIL to an even harder fine manipulation challenge: using chopsticks to grasp a coin and then insert it into the thin slot in a piggy bank (as shown in Fig. 2). It is very challenging to collect demonstration data. To make the task simpler, we randomize the coin's initial position in a small, palm-sized region on the table. With 88 successful demonstrations collected by a heuristic controller and the enhanced robustness provided by CCIL, we were able to improve the behavior cloning success rate from 90% to near 100%. On a lower data region (18 successful demonstrations), CCIL could boost BC success from 33% to about 66%. Our experiments validate the effectiveness of applying CCIL to boost the performance of state-of-the-art imitation learning methods for real-world fine manipulation tasks, especially in low-data regimes.



(a) Applying CCIL provides a statistically significant performance boost to vanilla behavior cloning.

(b) Boost to diffusion policies.

Figure 8. CCIL generally provides a statistically significant performance boost for imitation learning methods, especially in low-data regimes. We use asterisks to denote statistical significance: $* : p < 0.1$, $** : p < 0.05$, $*** : p < 0.01$, and $**** : p < 0.001$. ns denotes $p \geq 0.1$. (a) showcases the performance boost after applying CCIL to vanilla BC policies on three tasks across various dataset sizes. (b) showcases the performance boost after applying CCIL to diffusion policies on the Coin task across various dataset sizes.



(a) BC+CCIL (top) grasps the cube while BC (bottom) knocks it over.

(b) BC+CCIL (top) inserts the gear while BC (bottom) misses the hole.

(c) BC+CCIL (top) picks up the coin while BC (bottom) misses.

Figure 9. Comparison of BC+CCIL (top) and BC (bottom) across fine-manipulation tasks.

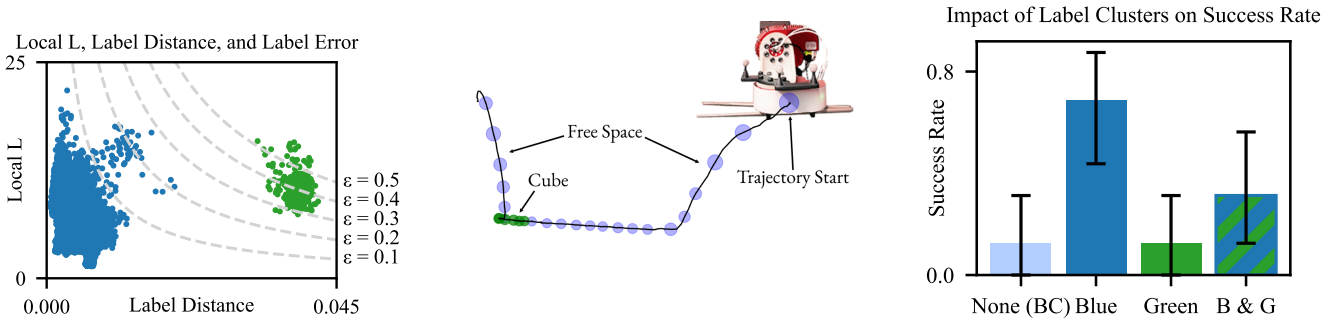
7.2 Resilience of CCIL to Presence of Local Discontinuity

H.2 investigates whether CCIL's local continuity assumptions are valid and realistic to be applied to real-world fine manipulation tasks.

Real-world tasks contain local Lipschitz continuity that can be realized by learning dynamics models. In training the dynamics model, we experiment with different Lipschitz constraints, measure the resulting models' local Lipschitz coefficient and plot the average of the coefficient in Fig. 11. With a large upper bound on the Lipschitz constraint (loose constraint), the local coefficients increase but converge to that of an unconstrained model. These findings imply that our environments exhibit local continuity that the learned dynamics models are fitting to, even without explicitly enforcing the Lipschitz continuity coefficient.

The learned dynamics model and generated labels have varying continuity that correlates with the real world. For each generated label, we can measure the local Lipschitz coefficients using the learned dynamics model ("Local L"), as described in Remark 4.4. We generate a scatterplot for the generated labels in Fig. 10(a). We see two clusters (green and blue). We then sample some of the labels for visualization and plot them along an expert trajectory in Fig. 10(b). We see the green cluster correspond to labels generated near the cube and the blue labels are mostly in free space when the robot moves without collision. Intuitively, the learned model and generated label have higher errors for the green group that contains discontinuity.

We test how the policy performance changes as we incorporate (1) blue labels, (2) green labels, and (3) both labels into the demonstration dataset, as shown in Fig. 10(c). Training with blue labels improved the agent's success rate. However, training with the green labels associated with



(a) Clusters reveal dynamics regimes.

(b) High label rejection clusters around contact.

(c) Label qualities impact on imitation learning policy performance.

Figure 10. Using a learned dynamics model to measure the local Lipschitz coefficients for each data point yields insights about the discontinuity around the data point. We visualize the estimated Lipschitz coefficients for the 20% dataset for the *GraspCube* task. (a) Plotting the estimated label error reveals two distinct clusters of corrective labels. (b) The green cluster mostly corresponds to labels where the cube is being manipulated, and the blue cluster mostly corresponds to arm free space motion. (c) Policies trained using just the blue cluster (low label rejection threshold) are more successful when compared with those trained with the green cluster (high label rejection threshold) or both.

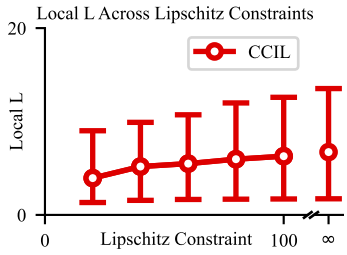


Figure 11. Dynamics model learning

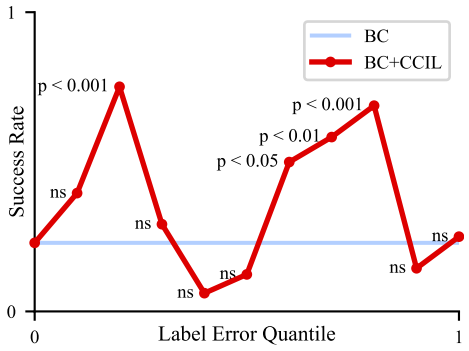


Figure 12. Policy performance when filtering out varying fractions of the generated labels by label error.

contact-rich states and higher errors is not as useful. Such labels actually hinder performance. This highlights how label quality can impact CCIL performance and the significance of filtering generated labels.

7.3 Impact of Quality of Generated Labels

The label filtering threshold controls what generated labels are used for when training a policy. It is crucial to understand its effects and how to tune it properly. We proposed a practical way to enforce filtering threshold via label rejection quantile, **H.3** explores how this design choice affects CCIL's performance.

We choose the GraspCube task with 20% of the data, or 100 trajectories. We evaluate running CCIL with different label rejection quantiles and report the success rate in Fig. 12. The rejection quantile controls how many generated labels are used (25%, 75%, etc) based on their computed error bound.

Low-quality labels can harm policy performance. Using all generated labels (choosing a quantile close or equal to 1) makes the performance of CCIL similar to BC or worse. With this quantile, the generated labels have high error bounds and could be incorrect under the true dynamics. If these labels are included when training the policy, they could potentially hinder policy performance.

Need to balance label error and usefulness for CCIL to succeed. Labels with lower error bounds are more trustworthy. However, these labels tend to be closer to the demonstration data (small label distance). Including labels with conservative error bounds may fail to expand the data support. Conversely, a label that is further from the demonstration data can expand the data support, but potentially introduces a higher associated error. Fig. 12 shows that intermediate values of the label error quantile achieve a higher success rate, indicating that one needs to balance between label accuracy and usefulness to maximize CCIL's performance. Notably, however, while empirically there are strong peaks in performance in the middle, it is difficult to predict a priori where those peaks may be, pointing to the sensitivity of CCIL to this filtering threshold.

7.4 Impact of Continuity Constraint

H.4 asks how enforcing Lipschitz continuity for training the dynamics model impacts CCIL's performance.

We train the dynamics model using different Lipschitz constraints; For each model, we generate labels and test the performance of CCIL with various label error quantiles. All hyperparameters other than those ablated upon are fixed. We illustrate the resulting success rate in Fig. 13 and cross the cells whose performance are not statistically significant compared to BC.

CCIL is robust to different Lipschitz constraints. Interestingly, we see that for each Lipschitz constraint, there is a label rejection quantile that achieves a significant boost in performance over BC. This indicates that CCIL could work well for a wide range of constraints, making it robust to this parameter.

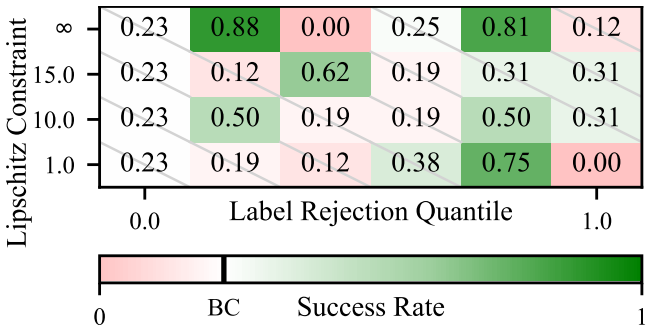


Figure 13. Hyperparameter ablation for CCIL, varying the Lipschitz constraint and the label rejection quantile. Green cells indicate a performance boost over BC, while red cells are worse. Crossed out cells are not significant at the $p < 0.05$ level, while the others are.

CCIL can work with an unconstrained dynamics model. Surprisingly, even using an unconstrained dynamics model, indicated by the Lipschitz constraint of ∞, CCIL still achieves good performance with certain quantile. We investigate this phenomenon in Fig. 11 and observe that the unconstrained model still exhibits local continuity over the demonstration data, as shown by the relatively small local Lipschitz coefficients across the dataset. Interestingly, we observe that as the Lipschitz constraint increases, the distribution of local Lipschitz coefficients converges to that of the unconstrained model.

These findings together imply that in environments with sufficient local continuity, explicit Lipschitz constraint enforcement may be unnecessary and tuning the label rejection quantile can be more critical. When applying CCIL to new environments, one can start with unconstrained dynamics model for their simplicity.

8 Discussion

In this work, we introduce a novel framework that leverages local continuity in environmental dynamics to train dynamic functions from expert demonstrations and generate corrective labels. These generated labels encourage imitation learning agents to stay within the support of the demonstration data. Our key insight is to exploit local continuity in the environment, which provides theoretical guarantees on where to trust the learned dynamics models beyond the data support. We demonstrate the efficacy of our approach across diverse simulation tasks with varying state representations and action spaces, and conduct extensive evaluations and ablations on multiple fine-manipulation tasks on a real robotic platform. Our experiments empirically validate that the assumption of local continuity holds even in challenging real-world scenarios, providing confidence in future application of our method. We also demonstrate CCIL's applicability to different policy classes, including state of the art diffusion policies.

However, our method is not without limitations. One challenge is to train better dynamics models from expert demonstrations, which is an open research frontier. Another challenge is applying our core assumption of local Lipschitz continuity in the dynamics to domains with highly discontinuous representations, such as pixel-based image spaces. Despite these limitations, our work paves the way for many exciting avenues of future research in model-based imitation learning, aiming to overcome these challenges and expand the scope of CCIL.

8.1 Learning Dynamics Models for Imitation Learning

We are not the first to use learned dynamics models to combat covariate shift in offline imitation learning. Other works such as Chang et al. (38) mark a family of methods that aim to leverage model learning to aid in the imitation learning setting. We, however, are the first to exploit the presence of local continuity in the physics while also relying solely on expert demonstrations, which have very limited coverage.

Whereas prior works have used additional offline demonstrations to increase data coverage and therefore accuracy of the learned model, we seek to exploit structure in the environment instead of requiring extra data. We reason that continuity in the environment can provide similar benefits as the supplemental offline dataset, due to our derived label error bounds. We have shown that a relatively simple method of model learning, namely a MLP architecture with Lipschitz regularization, works well and generates accurate labels near the expert data, but if future work can increase the accuracy of learned models even further from the training data, it would allow CCIL to increase state coverage even more. Alternative model architectures or continuity regularization techniques could be a fruitful direction of future exploration, and may unlock better out-of-distribution performance of the learned model.

Beyond our usage of corrective labels, efficiently learning accurate dynamics models from offline data remains an opportunity for exciting new avenues of research. Learned models allow data-driven methods to build an understanding of the environment, which we can exploit to build powerful and robust robotic agents. Further development on offline model learning can unlock powerful methods with minimal requirements and assumptions, greatly increasing robotic capabilities in the real world.

8.2 Discontinuous State Representations

CCIL works well in real robotic tasks by exploiting local continuity in system dynamics. However, its key assumption relies on having access to a state representation with locally continuous properties. Images and other high-dimensional representations can exhibit varying discontinuities with respect to the dynamics, which can limit CCIL's applicability to these problem settings.

We posit that there can exist local continuity in the underlying system dynamics despite the representation being discontinuous. If we can extract a latent representation with sufficient local continuity from the raw observation, it would enable the application of CCIL on the latent space. For example, learning embeddings for image-based robotic tasks have been widely studied (64; 65) and one can employ similar strategies for extracting latent representations. This can be an exciting direction for future research exploring how learned pixel-based dynamics model can be used for data augmentation.

Learning a latent dynamics model while enforcing local continuity remains an open research question. Solutions can enable the application of CCIL and greatly increase its applicability. Additionally, it would further our understanding of learning dynamics models for complex environments with highly discontinuous dynamics where low-dimensional states cannot be easily estimated, paving the way for other model-based methods (38; 36) to make robotics learning algorithms more robust and efficient.

REFERENCES

[1] H. Ravichandar, A. S. Polydoros, S. Chernova, and A. Billard, "Recent advances in robot learning from demonstration," *Annual review of control, robotics, and autonomous systems*, vol. 3, no. 1, pp. 297–330, 2020.

[2] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang *et al.*, "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.

[3] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit behavioral cloning," in *Conference on Robot Learning*. PMLR, 2022, pp. 158–168.

[4] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.

[5] P. de Haan, D. Jayaraman, and S. Levine, "Causal confusion in imitation learning," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, H. M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. B. Fox, and R. Garnett, Eds., 2019, pp. 11 693–11 704. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/947018640bf36a2bb609d3557a285329-Abstract.html

[6] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "Dart: Noise injection for robust imitation learning," in *Conference on Robot Learning*. PMLR, 2017, pp. 143–156.

[7] P. Florence, L. Manuelli, and R. Tedrake, "Self-supervised correspondence in visuomotor policy learning," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 492–499, 2019.

[8] A. Zhou, M. J. Kim, L. Wang, P. Florence, and C. Finn, "Nerf in the palm of your hand: Corrective augmentation for robotics via novel-view synthesis," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 17 907–17 917.

[9] L. Ke, J. Wang, T. Bhattacharjee, B. Boots, and S. Srinivasa, "Grasping with chopsticks: Combating covariate shift in model-free imitation learning for fine manipulation," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 6185–6191.

[10] D. A. Pomerleau, "ALVINN: an autonomous land vehicle in a neural network," *Advances in Neural Information Processing Systems*, vol. 1, 1988.

[11] D. Silver, J. Bagnell, and A. Stentz, "High performance outdoor navigation from overhead data using imitation learning," *Robotics: Science and Systems IV, Zurich, Switzerland*, vol. 1, 2008.

[12] R. Rahmatizadeh, P. Abolghasemi, L. Bölöni, and S. Levine, "Vision-based multi-task manipulation for inexpensive robots using end-to-end learning from demonstration," in *2018 IEEE International Conference on Robotics and Automation*. IEEE, 2018.

[13] Z. Q. Chen, K. Van Wyk, Y.-W. Chao, W. Yang, A. Mousavian, A. Gupta, and D. Fox, "Learning robust real-world dexterous grasping policies via implicit shape augmentation," *2022 Conference on Robot Learning*, 2022.

[14] G. Zhou, L. Ke, S. Srinivasa, A. Gupta, A. Rajeswaran, and V. Kumar, "Real world offline reinforcement learning with realistic data source," *2023 International Conference on Robotics and Automation*, 2023.

[15] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, 2024.

[16] L. Ke, Y. Zhang, A. Deshpande, S. Srinivasa, and A. Gupta, "Ccil: Continuity-based data augmentation for corrective imitation learning," *2024 International Conference on Learning Representations*, 2023.

[17] A. Deshpande, L. Ke, Q. Pfeifer, A. Gupta, and S. S. Srinivasa, "Data efficient behavior cloning for fine manipulation via continuity-based corrective labels," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 8531–8538.

[18] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," *Robotics: Science and Systems*, 2023.

[19] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, J. Peters *et al.*, "An algorithmic perspective on imitation learning," *Foundations and Trends® in Robotics*, vol. 7, no. 1-2, pp. 1–179, 2018.

[20] T. Pearce, T. Rashid, A. Kanervisto, D. Bignell, M. Sun, R. Georgescu, S. V. Macua, S. Z. Tan, I. Momennejad, K. Hofmann *et al.*, "Imitating human behaviour with diffusion models," *arXiv preprint arXiv:2301.10677*, 2023.

[21] K. Kawaharazuka, T. Matsushima, A. Gambardella, J. Guo, C. Paxton, and A. Zeng, "Real-world robot applications of foundation models: A review," *Advanced Robotics*, vol. 38, no. 18, pp. 1232–1254, 2024.

[22] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, "π_0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[23] M. Shu, Y. Shen, M. C. Lin, and T. Goldstein, "Adversarial differentiable data augmentation for autonomous systems," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 14 069–14 075.

[24] A. Rangesh, N. Deo, R. Greer, P. Gunaratne, and M. M. Trivedi, "Predicting take-over time for autonomous driving with real-world data: Robust data augmentation, models, and evaluation," *arXiv preprint arXiv:2107.12932*, 2021.

[25] A. Venkatraman, M. Hebert, and J. A. Bagnell, "Improving multi-step prediction of learned time series models," in *Twenty-Ninth AAAI Conference on Artificial Intelligence*,

2015.

[26] J. C. Spencer, S. Choudhury, A. Venkatraman, B. D. Ziebart, and J. A. Bagnell, "Feedback in imitation learning: The three regimes of covariate shift," *CoRR*, vol. abs/2102.02872, 2021. [Online]. Available: https://arxiv.org/abs/2102.02872

[27] A. Block, A. Jadbabaie, D. Pfrommer, M. Simchowitz, and R. Tedrake, "Provable guarantees for generative behavior cloning: Bridging low-level stability and high-level behavior," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[28] J. Y. Park and L. Wong, "Robust imitation of a few demonstrations with a backwards model," *Advances in Neural Information Processing Systems*, vol. 35, pp. 19 759–19 772, 2022.

[29] K.-W. Chang, A. Krishnamurthy, A. Agarwal, H. Daumé III, and J. Langford, "Learning to search better than your teacher," in *International Conference on Machine Learning*, 2015.

[30] W. Sun, A. Venkatraman, G. J. Gordon, B. Boots, and J. A. Bagnell, "Deeply aggrevated: Differentiable imitation learning for sequential prediction," in *International Conference on Machine Learning*. PMLR, 2017, pp. 3309–3318.

[31] J. Ho and S. Ermon, "Generative adversarial imitation learning," *NeurIPS*, vol. 29, 2016.

[32] S. Reddy, A. D. Dragan, and S. Levine, "Sqil: Imitation learning via reinforcement learning with sparse rewards," *2020 International Conference on Learning Representations*, 2020.

[33] J. Fu, K. Luo, and S. Levine, "Learning robust rewards with adversarial inverse reinforcement learning," *2018 International Conference on Learning Representations*, 2018.

[34] L. Ke, S. Choudhury, M. Barnes, W. Sun, G. Lee, and S. Srinivasa, "Imitation learning as f-divergence minimization," in *Algorithmic Foundations of Robotics XIV: Proceedings of the Fourteenth Workshop on the Algorithmic Foundations of Robotics 14*. Springer, 2021, pp. 313–329.

[35] G. Swamy, S. Choudhury, J. A. Bagnell, and S. Wu, "Of moments and matching: A game-theoretic framework for closing the imitation gap," in *International Conference on Machine Learning*. PMLR, 2021, pp. 10 022–10 032.

[36] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "Morel: Model-based offline reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 21 810–21 823.

[37] A. Reichlin, G. L. Marchetti, H. Yin, A. Ghadirzadeh, and D. Kragic, "Back to the manifold: Recovering from out-of-distribution states," in *International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022.

[38] J. Chang, M. Uehara, D. Sreenivas, R. Kidambi, and W. Sun, "Mitigating covariate shift in imitation learning via offline data with partial coverage," *2021 Advances in Neural Information Processing Systems*, vol. 34, pp. 965–979, 2021.

[39] B. Bonnard, J.-B. Caillau, and E. Trélat, "Second order optimality conditions in the smooth case and applications in optimal control," *ESAIM: Control, Optimisation and Calculus of Variations*, vol. 13, no. 2, pp. 207–236, 2007.

[40] W. Li and E. Todorov, "Iterative linear quadratic regulator design for nonlinear biological movement systems." in *ICINCO (1)*. Citeseer, 2004, pp. 222–229.

[41] D. Seto, A. M. Annaswamy, and J. Baillieul, "Adaptive control of nonlinear systems with a triangular structure," *IEEE Transactions on Automatic Control*, vol. 39, no. 7, pp. 1411–1428, 1994.

[42] N. E. Kahveci, "Robust adaptive control for unmanned aerial vehicles," Ph.D. dissertation, University of Southern California, 2007.

[43] J. Sarangapani, *Neural Network Control of Nonlinear Discrete-Time Systems*. CRC press, 2018.

[44] S. M. Khansari-Zadeh and A. Billard, "Learning stable nonlinear dynamical systems with gaussian mixture models," *IEEE Transactions on Robotics*, vol. 27, no. 5, pp. 943–957, 2011.

[45] N. B. Figueroa Fernandez and A. Billard, "A physically-consistent bayesian non-parametric mixture model for dynamical system learning," *Proceedings of Machine Learning Research*, 2018.

[46] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," *2020 International Conference on Learning Representations*, 2020.

[47] T. Wang, S. S. Du, A. Torralba, P. Isola, A. Zhang, and Y. Tian, "Denoised mdps: Learning world models better than the world itself," *2022 International Conference on Machine Learning*, 2022.

[48] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, 2023.

[49] K. Asadi, D. Misra, and M. L. Littman, "Lipschitz continuity in model-based reinforcement learning," in *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, ser. Proceedings of Machine Learning Research, J. G. Dy and A. Krause, Eds., vol. 80. PMLR, 2018, pp. 264–273. [Online]. Available: http://proceedings.mlr.press/v80/asadi18a.html

[50] P. L. Bartlett, D. J. Foster, and M. J. Telgarsky, "Spectrally-normalized margin bounds for neural networks," *NeurIPS*, vol. 30, 2017.

[51] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *International Conference on Machine Learning*. PMLR, 2017, pp. 214–223.

[52] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," *2018 International Conference on Learning Representations*, 2018.

[53] G. Shi, X. Shi, M. O'Connell, R. Yu, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, "Neural lander: Stable drone landing control using learned dynamics," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 9784–9790.

[54] S. Pfrommer, M. Halm, and M. Posa, "Contactnets: Learning discontinuous contact dynamics with smooth, implicit representations," in *Conference on Robot Learning*. PMLR, 2021, pp. 2279–2291.

[55] K. Khetarpal, Z. Ahmed, G. Comanici, D. Abel, and D. Precup, "What can i do here? a theory of affordances in reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2020, pp. 5243–5253.

[56] A. Zhang, R. McAllister, R. Calandra, Y. Gal, and S. Levine, "Learning invariant representations for reinforcement learning

without reconstruction," *2021 International Conference on Learning Representations*, 2021.

[57] C. Zhu, M. Simchowitz, S. Gadipudi, and A. Gupta, "Repo: Resilient model-based reinforcement learning by regularizing posterior predictability," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 32 445–32 467.

[58] T. Wang, X. Bao, I. Clavera, J. Hoang, Y. Wen, E. Langlois, S. Zhang, G. Zhang, P. Abbeel, and J. Ba, "Benchmarking model-based reinforcement learning," *arXiv preprint arXiv:1907.02057*, 2019.

[59] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville, "Improved training of wasserstein gans," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[60] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.

[61] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig, "Learning to fly—a gym environment with pybullet physics for reinforcement learning of multi-agent quadcopter control," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 7512–7519.

[62] L. Ke, A. Kamat, J. Wang, T. Bhattacharjee, C. Mavrogiannis, and S. S. Srinivasa, "Telemanipulation with chopsticks: Analyzing human factors in user demonstrations," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 11 539–11 546.

[63] S. Gubbi, S. Kolathaya, and B. Amrutur, "Imitation learning for high precision peg-in-hole tasks," in *ICCAR*, 2020.

[64] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, "R3m: A universal visual representation for robot manipulation," 2022.

[65] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," 2021.

[66] M. A. Hearst, S. T. Dumais, E. Osuna, J. Platt, and B. Scholkopf, "Support vector machines," *IEEE Intelligent Systems and Their Applications*, vol. 13, no. 4, pp. 18–28, 1998.

[67] N. D. Ratliff, J. A. Bagnell, and M. A. Zinkevich, "Maximum margin planning," in *Proceedings of the 23rd international conference on Machine learning*, 2006, pp. 729–736.

[68] F. D. d. R. Oliveira, E. L. O. Batista, and R. Seara, "On the compression of neural networks using 0-norm regularization and weight pruning," *Neural Networks*, vol. 171, pp. 343–352, 2021.

[69] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4rl: Datasets for deep data-driven reinforcement learning," *arXiv preprint arXiv:2004.07219*, 2020.

A Generating Corrective Labels with Known Dynamics Function

With a known dynamics function, we show an example algorithm that one can use to generate N corrective labels (as defined in Sec 4.1). The algorithm first trains a behavior cloning policy, samples test time roll-out trajectories from the learned policy, and then derives labels using a root-finding solver.

Algorithm 2: Generating Corrective Labels using Dynamics Function

Input Expert Data $\mathcal{D}* = (s_i^*, a_i^*, s_{i+1}^*)$.
Input Dynamics function $f(s^{i+1}|s^i, a^i)$.
Input Parameter N.
Initialize $\mathcal{D}^{\mathcal{G}} \leftarrow \varnothing, \mathcal{S}' \leftarrow \varnothing$
$\hat{\pi} = \arg\min_{\hat{\pi}} -\mathbb{E}_{s_i^*, a_i^*, s_{i+1}^* \sim \mathcal{D}^*} \log(\hat{\pi}(a_i^* \mid s_i^*))$
for $i \in 1..N$ **do**
    $s_0^{\mathcal{G}} \sim P_0, s_{j+1}^{\mathcal{G}} \sim f(s_j^{\mathcal{G}}, \pi(s_j^{\mathcal{G}})), \mathcal{S}' \leftarrow \mathcal{S}' \cup \{s_j^{\mathcal{G}}\}$
end for
for $i \in 1..N$ **do**
    $a_i^{\mathcal{G}} \leftarrow \arg\min_{a^{\mathcal{G}}} ||[s_i^{\mathcal{G}} + f(s_i^{\mathcal{G}}, a_i^{\mathcal{G}})] - s_k^*||$
    $\mathcal{D}^{\mathcal{G}} \leftarrow \mathcal{D}^{\mathcal{G}} \cup (s_i^{\mathcal{G}}, a_i^{\mathcal{G}})$
end for
return $\mathcal{D}^{\mathcal{G}}$

B Proofs

B.1 Proof of Theorem 4.2

Notation. Let f be the ground truth 1-step residual dynamics model, and let $\hat{f}$ be the learned approximation of f. Given a demonstration (s_{t+1}^*, a_{t+1}^*), we have generated a corrective label $(s^{\mathcal{G}}, a_{t+1}^*)$.

Assumptions:

1. The estimation error of the learned dynamics model *at the training data* is bounded.
   $\left\| f(s_{t+1}^*, a_{t+1}^*) - \hat{f}(s_{t+1}^*, a_{t+1}^*) \right\| \leq \epsilon$.
2. $\hat{f}$ is locally K_1-Lipschitz in a neighborhood $\widehat{U}$ around (s_{t+1}^*, a_{t+1}^*). i.e., for $s_t^{\mathcal{G}} \in \widehat{U}$:
   $\left\| \hat{f}(s_t^{\mathcal{G}}, a_{t+1}^*) - \hat{f}(s_{t+1}^*, a_{t+1}^*) \right\| \leq K_1 \left\| s_t^{\mathcal{G}} - s_{t+1}^* \right\|$
3. f is locally K_2-Lipschitz in a neighborhood U around (s_{t+1}^*, a_{t+1}^*). i.e., for $s_t^{\mathcal{G}} \in U$:
   $\left\| f(s_t^{\mathcal{G}}, a_{t+1}^*) - f(s_{t+1}^*, a_{t+1}^*) \right\| \leq K_2 \left\| s_t^{\mathcal{G}} - s_{t+1}^* \right\|$
4. $s_t^{\mathcal{G}}$ is within the region of local continuity around s_{t+1}^* for both f and $\hat{f}$:
   $s_t^{\mathcal{G}} \in U_{s_{t+1}^*} \cap \widehat{U}_{s_{t+1}^*}$.

Proof:

$$
\begin{aligned}
&\left\| f(s_t^{\mathcal{G}}, a_{t+1}^*) - \hat{f}(s_t^{\mathcal{G}}, a_{t+1}^*) \right\| \\
&= \left\| f(s_t^{\mathcal{G}}, a_{t+1}^*) - f(s_{t+1}^*, a_{t+1}^*) + f(s_{t+1}^*, a_{t+1}^*) - \hat{f}(s_{t+1}^*, a_{t+1}^*) + \hat{f}(s_{t+1}^*, a_{t+1}^*) - \hat{f}(s_t^{\mathcal{G}}, a_{t+1}^*) \right\| \\
&\leq \left\| f(s_t^{\mathcal{G}}, a_{t+1}^*) - f(s_{t+1}^*, a_{t+1}^*) \right\| + \left\| f(s_{t+1}^*, a_{t+1}^*) - \hat{f}(s_{t+1}^*, a_{t+1}^*) \right\| + \left\| \hat{f}(s_{t+1}^*, a_{t+1}^*) - \hat{f}(s_t^{\mathcal{G}}, a_{t+1}^*) \right\| \\
&\leq \epsilon + (K_1 + K_2) \left\| s_t^{\mathcal{G}} - s_{t+1}^* \right\|
\end{aligned}
$$

Remark Our assumption (1) about the error of the learned dynamics model is not a global constraint but simply requires the model to have a small prediction error *at the specific data point*. Our assumptions (2) and (3) about the size of the local Lipschitz continuity is not a requirement on the global Lipschitz continuity in the dynamics, but simply requires space *near the expert support* to exhibit local continuity. Our proof leverages simple triangle inequality and is valid only near expert data support.

B.2 Proof of Theorem 4.3

Notation. Let f be the ground truth 1-step residual dynamics model, and let $\hat{f}$ be the learned approximation of f. Given a demonstration $(s_t^*, a_t^*, s_{t+1}^*)$, we have generated a corrective label $(s^{\mathcal{G}}, a_t^* + \Delta, s_{t+1}^*)$.

Assumptions

1. f is locally K_S-Lipschitz in *state* in a neighborhood U_S around (s_t^*, a_t^*).
   i.e., for $s_t^{\mathcal{G}} \in U_S$:
   $\left\|f(s_t^{\mathcal{G}}, a_t^*) - f(s_t^*, a_t^*)\right\| \leq K_S \left\|s_t^{\mathcal{G}} - s_t^*\right\|$.
2. f is locally K_A-Lipschitz in *action* in a neighborhood U_A around $(s_t^{\mathcal{G}}, a_t^*)$.
   i.e., for $a_t^* + \Delta \in U_A$:
   $\left\|f(s_t^{\mathcal{G}}, a_t^*) - f(s_t^{\mathcal{G}}, a_t^* + \Delta)\right\| \leq K_A \left\|\Delta\right\|$.
3. Using rejection sampling, we can enforce $\left\|s_t^{\mathcal{G}} - s_t^*\right\| \leq \epsilon_{rej}$.
4. Given that the generated labels come from a root solver:
$$\begin{aligned} s_{t+1}^* - \hat{f}(s_t^{\mathcal{G}}, a_t^* + \Delta) - s_t^{\mathcal{G}} &= \epsilon_{opt} \text{ where } \epsilon_{opt} \to 0 \\ s_t^* + f(s_t^*, a_t^*) - \hat{f}(s_t^{\mathcal{G}}, a_t^* + \Delta) - s_t^{\mathcal{G}} &= \epsilon_{opt} \\ f(s_t^*, a_t^*) - \hat{f}(s_t^{\mathcal{G}}, a_t^* + \Delta) &= s_t^{\mathcal{G}} - s_t^* + \epsilon_{opt} \end{aligned}$$
5. The corrective label is within the neighborhood of local Lipschitz continuity of f:
   $s^{\mathcal{G}} \in U_S$ and $a_t^* + \Delta \in U_A$

Proof

$$\begin{aligned} &\left\|f(s_t^{\mathcal{G}}, a_t^* + \Delta) - \hat{f}(s_t^{\mathcal{G}}, a_t^* + \Delta)\right\| \\ &= \left\|f(s_t^{\mathcal{G}}, a_t^* + \Delta) - f(s_t^{\mathcal{G}}, a_t^*) + f(s_t^{\mathcal{G}}, a_t^*) - f(s_t^*, a_t^*) + f(s_t^*, a_t^*) - \hat{f}(s_t^{\mathcal{G}}, a_t^* + \Delta)\right\| \\ &\leq \left\|f(s_t^{\mathcal{G}}, a_t^* + \Delta) - f(s_t^{\mathcal{G}}, a_t^*)\right\| + \left\|f(s_t^{\mathcal{G}}, a_t^*) - f(s_t^*, a_t^*)\right\| + \left\|f(s_t^*, a_t^*) - \hat{f}(s_t^{\mathcal{G}}, a_t^* + \Delta)\right\| \\ &\leq K_A \left\|\Delta\right\| + K_S \left\|s_t^{\mathcal{G}} - s_t^*\right\| + \left\|s_t^{\mathcal{G}} - s_t^*\right\| + ||\epsilon_{opt}|| \\ &\leq K_A \left\|\Delta\right\| + (1 + K_S) \cdot \left\|s_t^{\mathcal{G}} - s_t^*\right\| + ||\epsilon_{opt}|| \end{aligned}$$

When the root solver yields a solution with $||\epsilon_{opt}|| \to 0$ and we apply rejection sampling to enforce $\left\|s_t^{\mathcal{G}} - s_t^*\right\| \leq \epsilon_{rej}$, we have the error bounded by $K_A \left\|\Delta\right\| + (1 + K_S) \cdot \epsilon_{rej}$

C Details for CCIL

Our proposed framework for generating corrective labels, **CCIL**, takes three steps:

1. **Learn a dynamics model**: fit a dynamics model $\hat{f}$ that is locally Lipschitz continuous.
2. **Generate labels**: solve a root-finding equation in Sec. 4.3 to generate labels.
3. **Augment the dataset and train a policy**: We use behavior cloning for simplicity to train a policy.

C.1 Learning a dynamics function while enforcing local Lipschitz continuity

There are many function approximators to learn a model. For example, using Gaussian process can produce a smooth dynamics model but might have limited scalability when dealing with large amount of data. In this paper we demonstrate examples of using a neural network to learn the dynamics model. There are multiple ways to enforce Lipschitz continuity on the learned dynamics function, with varying levels of strength that trade off theoretical guarantees and learning ability. In Sec. 4.2 we discuss a simple penalty to enforce local Lipschitz continuity, here we provide a few alternative methods, including the one used by (53) and the one inspired by (59).

Global Lipschitz Continuity via Spectral Normalization. Using spectral norm with coefficient K provides the strongest guarantee that the dynamic model is global K-Lipschitz. Concretely, spectral normalization (52) normalizes the weights of the neural network following each gradient update. (53) used this method to train a dynamics function for drone navigation. Parameterizing $\hat{f}$ as a n-layer neural network with weight matrices $W_1, \ldots, W_n$, the learning objective becomes:

$$\arg\min_{\hat{f}} E_{s_j^*, a_j^*, s_{j+1}^* \sim \mathcal{D}^*} \left[\text{MSE}\right] \quad \text{while} \quad W_i \leftarrow \frac{W_i}{\max(\|W_i\|_2, K^{-n})} \cdot K^{-n} \tag{14}$$

Local Lipschitz Continuity via Sampling-based Penalty. Following (59), we can relax the global Lipschitz continuity constraint by penalizing any violation of the local Lipschitz constraint. This objective is stated in Eq. 4, which we reiterate here.

$$\begin{aligned} &\arg\min_{\hat{f}} E_{s_j^*, a_j^*, s_{j+1}^* \sim \mathcal{D}^*} \left[\text{MSE} + \lambda \cdot \mathbb{1}\big(\hat{f}'(s_j^*, a_j^*) > K\big)\right] \\ &\hat{f}'(s_j^*, a_j^*) \approx \mathbb{E}_{\Delta_s \sim \mathcal{N}} \left[\frac{\|\hat{f}(s_j^* + \Delta_s, a_j^*) - \hat{f}(s_j^*, a_j^*)\|}{\|\Delta_s\|}\right] \end{aligned} \tag{15}$$

To estimate the local Lipschitz continuity, we can use a sampling-based method, $\mathbb{E}_{\Delta_s \sim \mathcal{N}} \max\left(\hat{f}'(s_j^* + \Delta_s, a_j^*) - K, 0\right)$, which is indicative of whether the local continuity constraint is violated for a given state-action pair. The sampling-based penalty perturbs the data points by some sampled noise and enforces the Lipschitz constraint between the perturbed data and the original using a penalty term in the loss function.

Doing so ensures that the approximate model is mostly K Lipschitz-bounded while being predictive of the transitions in the expert data. This approximate dynamics model can then be used to generate corrective labels.

Local Lipschitz Continuity via Slack-variable. We can allow for a small amount of discontinuity in the learned dynamics model in the same way that slack variables are modeled in optimization problems, e.g., SVM (66) or max-margin planning with slack variables (67). With these insights in mind, we can reformulate the dynamics model learning objective as learning models that maximize likelihood (MSE) while minimizing the Lipschitz constant $L(s_j^*, a_j^*)$.

$$\begin{aligned} \arg\min_{\hat{f}} \mathbb{E}_{s_j^*, a_j^*, s_{j+1}^* \sim \mathcal{D}^*} \text{MSE} + \lambda_j \cdot L(s_j^*, a_j^*) + ||\lambda_j - \bar{\lambda}||_0 \\ \text{where } ||\lambda_j - \bar{\lambda}||_0 \approx 1 - \exp\left(-\beta|\lambda_j - \bar{\lambda}|\right) \\ \text{and } L(s_j^*, a_j^*) = \mathbb{E}_{\Delta_s \sim \mathcal{N}} \max\left(\hat{f}'(s_j^* + \Delta_s, a_j^*) - K, 0\right). \end{aligned} \tag{16}$$

To account for small amounts of discontinuity, we introduce a state-dependent variable λ_j to allow violation of the Lipschitz continuity constraint. We minimize the number of non-zero entries in the slack variables (as noted by the L0 norm, $||\lambda_j - \bar{\lambda}||_0$) to ensure the model is otherwise as smooth as possible besides small amounts of local discontinuity. Building on Oliveira et al. (68), we can use a practical, differentiable approximation for the L0 norm method, since the L0 norm itself is non-differentiable. Equipped with these locally continuous learned dynamics models, we can then generate corrective labels for robustifying imitation learning.

Local Lipschitz Continuity via Weighted Loss. Since we are optimizing a continuous function approximators, if the data contain subsets of discontinuity, such regions are likely to induce high training loss. We can create a slack variable for each data point, λ_j, to down-weight the on those regions. To do so, we reformulate Eq. 14 to add a slack penalty. Taking σ as the Sigmoid function:

$$\arg\min_{\hat{f}} \mathbb{E}_{s_j^*, a_j^*, s_{j+1}^* \sim \mathcal{D}^*} \left[\sigma(\lambda_j) \cdot \text{MSE} + \sum \sigma(\lambda_j)\right] \quad \text{while} \quad W_i \leftarrow \frac{W_i}{\max(\|W_i\|_2, K^{-n})} \cdot K^{-n} \tag{17}$$

Intuitively, we expect that spectral normalization tends to work better for simpler environments where the ground truth dynamics are global Lipschitz with some reasonable L, whereas the soft constraint should be better suited for data regimes with more complicated dynamics and discontinuity. It remains a research question how best to train dynamics function for each domain and representation with different physical properties.

C.2 Generating corrective labels

In Sec. 4.3 we discuss two techniques to generate corrective labels. Depending on the structure of the application domain, one can choose to generate labels either by backtrack or by disturbed actions. Both techniques require solving root-finding equations (Eq. 6 and Eq. 8). To solve them, we can transform the objective to an optimization problem and apply gradient descent.

Eq. 6 specifies the root finding problem for backtrack labels.

$$s_t^* - \hat{f}(s_{t-1}^{\mathcal{G}}, a_t^*) - s_{t-1}^{\mathcal{G}} = 0.$$

Given s_t^*, a_t^* and the learned dynamics function $\hat{f}$, we need to solve for $s_t^{\mathcal{G}}$ that satisfies the equation. We can instead optimize for

$$\arg\min_{s_{t-1}^{\mathcal{G}}} ||s_t^* - \hat{f}(s_{t-1}^{\mathcal{G}}, a_t^*) - s_{t-1}^{\mathcal{G}}|| \tag{18}$$

Similarly, we can transform Eq. 8 to become

$$\arg\min_{s_t^{\mathcal{G}}} ||s_t^{\mathcal{G}} + \hat{f}(s_t^{\mathcal{G}}, a_t^* + \Delta) - s_{t+1}^*|| \tag{19}$$

With access to the trained model $\hat{f}$ and its gradient $\frac{\partial \hat{f}}{\partial s_t^{\mathcal{G}}}$, one can use any optimizer. For simplicity, we use the Backward Euler solver that apply an iterative update $s_t^{\mathcal{G}} \leftarrow s_t^{\mathcal{G}} - s \cdot \frac{\partial \hat{f}}{\partial s_t^{\mathcal{G}}}$ where s is a step size. We repeat the update until the objective is within a threshold $||s_t^{\mathcal{G}} + \hat{f}(s_t^{\mathcal{G}}, a_t^* + \Delta) - s_{t+1}^*|| \leq \epsilon_{opt}$.

C.3 Generating corrective action chunks

Some imitation learning methods output action chunks instead of singular actions. Formally, we define a deterministic action chunking policy as $\pi\colon \mathcal{S} \to \mathcal{A}^{T_a}$, for some action chunking horizon $T_a \in \mathbb{N}$. Given an expert trajectory $\{(s_i^*, a_i^*, s_{i+1}^*)\}_{i=1,\dots,T}$ of length T, we can construct a dataset of pairs of states and action chunks for training by chunking the trajectory, namely $\mathcal{D}^* = \{(s_i^*, (a_i^*, \dots, a_{i+T_a}^*)\}_{i=1,\dots,T-T_a}$. For each pair, we generate a synthetic action label $(s_i^{\mathcal{G}}, a_i^{\mathcal{G}}) = \text{LabelGen}(s_i^*, a_i^*, s_{i+1}^i)$, and then construct a synthetic action chunk label $\left(s_i^{\mathcal{G}}, (a_i^G, a_i^*, \dots, a_{i+T_a-1}^*)\right)$.

Some implementations may require entire trajectories as input, from which it constructs training labels by chunking. In this case, we represent each synthetic action chunk as a trajectory of length T_a by zero-padding the states, i.e. $\{(s_i^{\mathcal{G}}, a_i^{\mathcal{G}}), (0, a_i^*), \dots, (0, a_{i+T_a-a}^*)\}$. To prevent training on the inserted null observations, we must then set the observation chunk horizon as $T_o = 1$, the action chunk horizon as T_a, and the prediction horizon as $T_p = T_o + T_a - 1$, where T_o, T_a, T_p are defined as in Chi et al. (15).

C.4 Using the generated labels

There are multiple ways to use the generated corrective labels. We can augment the dataset with the generated labels and treat them as if they are expert demonstrations. For example, for all experiments conducted in this paper, we train behavior cloning agents using the augmented dataset. Optionally, one can favor the original expert demonstrations by assigning higher weights to their training loss. We omit this step for simplicity in this paper.

Alternatively, one can query a trained imitation learning policy with the generated labels and measure the difference between our generated action and the policy output. This difference can be used as an alternative metric to evaluate the robustness of imitation learning agents when encountering a subset of out-of-distribution states. However, for a query state, our proposal does not necessarily recover *all* possible corrective actions. We defer exploring alternative ways of leveraging the generated labels to future work.

D Simulation Warm-Up Experimental Details

We provide details to reproduce our simulation warm-up experiments, including environment specification, expert data, parameter tuning for our proposal and details about the baselines. We will also open-source the code and the configuration we use for each experiment, once the proposal is published.

D.1 Environment and Task Design

We warmed up by conducting simulation experiments on 4 different domains and 8 robots, including the pendulum, a drone, a car, four robots for locomotion and one robot arm for manipulation. We consider 14 tasks: the pendulum, a modified pendulum swing task with discontinuity, three drone navigation tasks (fly-through, circle, hover), one LiDar racing task on F1tenth, four MuJoCo tasks (Hopper, HalfCheetah, Ant, Walker2D) and 4 MetaWorld tasks (coffee-pull, button-press-topdown, coffee-push, drawer-close). The F1tenth, MuJoCo and Metaworld environments are from open source implementations, and the drone environment is from a slightly modified open source implementation, discussed below. We will also describe how we set up the Pendulum environment and how we modify it to test our method with discontinuity.

Pendulum Formulation. The pendulum environment asks a policy to swing a pendulum up to the vertical position by applying torque. The properties of the system are controlled by the constants g, the gravitational acceleration, and l, the length of the pendulum. In all experiments, we take $g = 9.81$ and $l = 1$.

A pendulum's state is characterized by θ, the current angle, and $\dot{\theta}$, the current angular velocity. To avoid any issues regarding angle representation, we do not directly store θ in the state representation; instead, we parameterize the state as $s = \begin{bmatrix}\sin\theta & \cos\theta & \dot{\theta}\end{bmatrix}^T$. A policy can control the system by applying torque to the pendulum, which we represent as a scalar a, which is clamped to the range $[-3, 3]$.

The continuous-time dynamics function is given by:

$$\frac{ds}{dt} = f(s, a) = \begin{bmatrix} \dot{\theta}\cos\theta \\ -\dot{\theta}\sin\theta \\ -\frac{g}{l}\sin\theta + a \end{bmatrix}.$$

This continuous dynamics model is then discretized to a timestep of 0.02 seconds using RK4. Additionally, although not required by the algorithms we study, we create the following reward function. where θ is the normalized pendulum angle in the range $[0, 2\pi)$ to metricize policy performance:

$$r(s, a) = -\frac{1}{2}\left\|\begin{bmatrix}\theta - \pi \\ \dot{\theta}\end{bmatrix}\right\|_2^2 - \frac{1}{2}a^2.$$

Expert Formulation for Pendulum. We formulate the expert policy using a combination of LQR and energy shaping control, where LQR is applied when the pendulum is near the top and energy-shaping is applied everywhere else. Note that

Environment	Trajectories
Pendulum	50
Discontinuous Pendulum	500
F1tenth Racing	1
Drone hover	5
Drone circle	30
Drone fly-through	50
MuJoCo - Ant	10
MuJoCo - Walker2D	20
MuJoCo - Hopper	25
MuJoCo - HalfCheetah	50
MetaWorld, all tasks	50

Table 2. Number of expert demonstration trajectories used in our experiments. We limit the amount of expert data to avoid making the task trivially solvable by naive behavior cloning.

the LQR gains were calculated by linearizing the dynamics around $\theta = \pi$, along with the cost function $c(s, a) = -r(s, a)$. So, the expert policy has the form:

$$\pi_e(s) = \begin{cases} -20.11(\theta - \pi) - 7.08\dot{\theta} & \text{if } |\theta - \pi| < 0.1 \\ -\dot{\theta}\left(\frac{1}{2}\dot{\theta}^2 - 9.81\cos\theta - 9.81\right) & \text{otherwise.} \end{cases}$$

Discontinuous Pendulum. We create a wall in the Pendulum environment to create local discontinuity. When the ball hits the wall, we *negate* its velocity, creating a discontinuity in the dynamics.

Drone. The drone environment used in our experiments is from a slightly modified fork of the original gym-pybullet-drones project. There are two notable modifications. Firstly, we added the circle task, where the agent aims to move in a circular trajectory. Second, we removed the floor, which better simulates a completely airborne drone and reduces training complexities caused by crashing into the floor.

D.2 Demonstration Data

To feed expert data to train imitation learning agents, we design expert policies for the pendulum. For all other environments, we use the expert data from the D4RL dataset (69). For drone environments, we first generate a bunch of via points alongside the target trajectories and then use a low-level PID controller to hit the via points one by one.

We note that it is possible to solve most tasks with naive behavior cloning if we feed them with a sufficient number of demonstrations. We thus limit the number of demonstrations we use for all tasks, as shown in Table. 2.

D.3 Baselines

We evaluate the following algorithms to gain a thorough understanding of how our proposal compares to relevant baselines.

- EXPERT: For reference, we plot the theoretical upper bound of our performance, which is the score achieved by an expert during data collection.
- BC: a naive behavior cloning agent that minimizes the KL divergence on a given dataset.
- NOISEBC: a modification to naive behavior cloning that injects a small disturbance noise to the input state and reuses the action label, as described in (9).
- MOREL: Morel (36) trains an ensemble of dynamics functions and uses the learned model for model-based reinforcement learning. It uses the variance between the model output as a proxy estimation of uncertainty to stay within high confidence region. We use the author implementation.
- MILO: MILO (38) learns a dynamic function from given offline dataset in an attempt to mitigate covariate shift. We use the author's implementation in our experiment. MILO traditionally requires a large batch of offline dataset to train a dynamics function, we use only the expert demo dataset to train a dynamics function.
- CCIL: our proposal to generate high-quality corrective labels.